



**Universitatea  
Transilvania  
din Brașov**  
FACULTATEA DE MATEMATICĂ  
ȘI INFORMATICĂ

Programul de studii:  
Informatică

## Lucrare de licență / disertație

**Favigon - Platformă web pentru proiectarea vizuală a interfețelor web, asistată de inteligență artificială, partajarea și conversia lor în cod**

**Autor:** Vasile Rareș Mihail  
**Coordonator științific:** Lect. dr. Spridon Delia-Elena

Brașov, 2026



# Rezumat

Lucrarea prezintă proiectarea și implementarea platformei *Favigon*, o aplicație web full-stack orientată spre construirea, editarea, partajarea și exportul interfețelor web. Ideea proiectului a pornit de la o observație simplă: în multe fluxuri de lucru actuale, trecerea de la design la cod rămâne fragmentată. Designerul lucrează într-un instrument, dezvoltatorul reconstruiește manual interfața, iar utilizatorul final vede rezultatul abia după mai multe iterații. În cazul proiectelor mici această separare este suportabilă, însă pentru proiecte dinamice devine rapid o sursă de timp pierdut, nealiniere și compromisuri tehnice.

În acest context, lucrarea urmărește realizarea unei platforme care să combine mai multe direcții într-un singur produs: editare vizuală într-un canvas, generare asistată de inteligență artificială, export de cod pentru mai multe framework-uri și o zonă socială în care proiectele pot fi publicate, apreciate și reutilizate. În forma actuală, Favigon se adresează în special designerilor, frontend developerilor, studenților și echipelor mici care vor să transforme rapid o idee de interfață într-un prototip editabil, într-un model intern coerent și, la final, în cod executabil. Din punct de vedere tehnic, aplicația folosește un frontend dezvoltat în Angular 20, un backend REST realizat în ASP.NET Core 9, o bază de date PostgreSQL și o infrastructură containerizată cu Docker Compose. Pentru expunerea aplicației în exterior este folosit și un tunel Cloudflare.

O parte importantă a lucrării este reprezentată de editorul canvas. Acesta nu este tratat ca o simplă interfață de desen, ci ca un subsistem complex, cu stare proprie, istoric de editare, persistare automată, operații de clipboard, gesturi de manipulare și suport pentru mai multe pagini. În jurul lui este construit un model de date orientat pe elemente și pagini, iar pe frontend logica este organizată în servicii dedicate pentru viewport, gesturi, istoric, persistare și generare de cod. Această separare menține codul relativ ușor de extins și evită concentrarea tuturor responsabilităților într-o singură componentă.

Capitolul central al lucrării este dedicat funcționalității de generare asistată de AI și conversiei din design în cod. În locul unei generări directe, greu de controlat, este folosită o soluție în trei faze. Prima fază extrage intenția utilizatorului și definește structura logică a paginii. A doua fază generează o reprezentare intermediară a interfeței, bazată pe un arbore de noduri cu informații de layout, stil, poziționare și metadate. A treia fază aplică stilizarea finală, păstrând structura deja validată. În acest mod, partea de AI poate fi constrânsă mai bine, iar rezultatele sunt mai ușor de verificat și reparat atunci când modelul produce ieșiri imperfecte.

Conversia în cod folosește aceeași reprezentare intermediară și o transformă în fișiere pentru HTML, React și Angular. Astfel, editorul și motorul de export nu sunt legate rigid de un singur framework. Din punct de vedere practic, această decizie a fost una dintre cele mai importante din proiect, deoarece a separat clar problema construirii designului de problema randării lui într-o tehnologie concretă. În plus, platforma suportă și export multi-page, precum și generare responsive prin diferențe între breakpoints.

Lucrarea tratează și componentele conexe care fac aplicația utilizabilă ca produs, nu doar ca demo tehnic: autentificare cu JWT în cookie-uri, OAuth prin furnizori externi, resetare parolă, autentificare în doi pași, gestiunea profilului, proiecte publice și private, explorare de proiecte și funcții sociale precum like, star, follow sau fork. În plus, sunt prezentate măsurile de securitate aplicate în backend, cum ar fi rate limiting-ul, limitarea dimensiunii cererilor, middleware-ul de tratare a excepțiilor și politicile de securitate HTTP.

Pentru validare, proiectul include atât testare automată, cât și scenarii funcționale. La momentul redactării lucrării, suita de teste backend conține 108 teste care verifică servicii, con-

trollere, middleware și motorul de conversie. Partea de frontend este testată mai puțin automatizat, motiv pentru care în lucrare sunt prezentate și validarea prin scenarii operaționale, observații directe și demonstrarea fluxurilor principale.

Rezultatul final este o platformă care nu încearcă să înlocuiască instrumentele consacrate din ecosistem, ci să ofere un flux integrat pentru proiectare rapidă, experimentare, export și publicare. Aportul personal se reflectă mai ales în integrarea coerentă a acestor module, în modelul intermediar folosit pentru design și cod, în pipeline-ul AI în trei faze și în modul în care editorul canvas a fost separat pe servicii specializate. În ansamblu, proiectul a permis combinarea unor teme de arhitectură software, dezvoltare web, securitate, UX și inteligență artificială aplicată într-un produs unitar.

# Cuprins

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>Rezumat</b>                                                | <b>i</b>  |
| <b>1 Introducere</b>                                          | <b>1</b>  |
| 1.1 Contextul actual al temei                                 | 1         |
| 1.2 Problema pe care o urmărește lucrarea                     | 1         |
| 1.3 Motivația alegerii temei                                  | 2         |
| 1.4 Obiectivele proiectului                                   | 2         |
| 1.5 Contribuțiile proprii                                     | 3         |
| 1.6 Metoda de lucru                                           | 3         |
| 1.7 Structura lucrării                                        | 4         |
| <b>2 Analiza domeniului și studiu comparativ</b>              | <b>5</b>  |
| 2.1 Evoluția spațiului design-to-code                         | 5         |
| 2.2 Criterii de comparație                                    | 5         |
| 2.3 Aplicații relevante din aceeași zonă                      | 6         |
| 2.3.1 Figma                                                   | 6         |
| 2.3.2 Webflow                                                 | 6         |
| 2.3.3 Framer                                                  | 6         |
| 2.3.4 Builder.io                                              | 6         |
| 2.4 Tabel comparativ                                          | 7         |
| 2.5 Poziționarea proiectului Favigon                          | 7         |
| 2.6 Golul pe care îl acoperă Favigon                          | 8         |
| 2.7 Concluzii intermediare                                    | 8         |
| <b>3 Fundamente teoretice și tehnologii utilizate</b>         | <b>9</b>  |
| 3.1 Considerații generale                                     | 9         |
| 3.2 Aplicații single-page și rolul frontend-ului modern       | 9         |
| 3.3 Arhitectura client-server și API-ul REST                  | 9         |
| 3.4 Autentificare, token-uri și securitate la nivel de cerere | 10        |
| 3.5 Reprezentarea intermediară a interfeței                   | 10        |
| 3.6 JSON Schema și ieșiri structurate pentru AI               | 11        |
| 3.7 Baza de date relațională și persistența                   | 11        |
| 3.8 Orchestrare și infrastructură                             | 12        |
| 3.9 De ce această combinație tehnologică a fost potrivită     | 13        |
| 3.10 Concluzii intermediare                                   | 13        |
| <b>4 Analiza și proiectarea sistemului</b>                    | <b>14</b> |
| 4.1 Pornirea de la cerințe                                    | 14        |
| 4.2 Cerințe funcționale                                       | 14        |
| 4.3 Cerințe nefuncționale                                     | 14        |
| 4.4 Actorii principali                                        | 15        |
| 4.5 Scenarii principale de utilizare                          | 16        |
| 4.5.1 Scenariul 1: creare și editare proiect                  | 16        |
| 4.5.2 Scenariul 2: generare asistată de AI                    | 16        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 4.5.3    | Scenariul 3: export în cod                                          | 16        |
| 4.5.4    | Scenariul 4: distribuire și reutilizare                             | 16        |
| 4.6      | Arhitectura generală                                                | 17        |
| 4.7      | Organizarea backend-ului pe straturi                                | 17        |
| 4.8      | Organizarea frontend-ului pe feature-uri                            | 18        |
| 4.9      | Modelul de date conceptual                                          | 18        |
| 4.10     | Fluxurile principale                                                | 20        |
| 4.10.1   | Fluxul de autentificare                                             | 20        |
| 4.10.2   | Fluxul de editare                                                   | 20        |
| 4.10.3   | Fluxul de export                                                    | 20        |
| 4.10.4   | Fluxul AI                                                           | 20        |
| 4.11     | Decizii de proiectare care au influențat cel mai mult implementarea | 20        |
| 4.12     | Concluzii intermediare                                              | 21        |
| <b>5</b> | <b>Implementarea platformei Favigon</b>                             | <b>22</b> |
| 5.1      | Privire de ansamblu asupra implementării                            | 22        |
| 5.2      | Rutare și pornirea aplicației frontend                              | 22        |
| 5.3      | Modulul de autentificare și profil                                  | 23        |
| 5.3.1    | Fluxurile de bază                                                   | 23        |
| 5.3.2    | JWT în cookie și refresh token                                      | 24        |
| 5.3.3    | Autentificarea în doi pași                                          | 24        |
| 5.3.4    | Profilul utilizatorului                                             | 24        |
| 5.4      | Gestionarea proiectelor                                             | 24        |
| 5.4.1    | Ciclul de viață al proiectelor                                      | 24        |
| 5.4.2    | Persistarea designului                                              | 24        |
| 5.4.3    | Thumbnail și asset-uri                                              | 25        |
| 5.5      | Funcționalități sociale și explorare                                | 25        |
| 5.6      | Suprafața API și distribuția responsabilităților                    | 26        |
| 5.7      | Editorul canvas                                                     | 27        |
| 5.7.1    | Rolul editorului în întregul produs                                 | 27        |
| 5.7.2    | Starea editorului                                                   | 27        |
| 5.7.3    | Viewport, selecție și gesturi                                       | 27        |
| 5.7.4    | Istoric și clipboard                                                | 28        |
| 5.7.5    | Managementul paginilor                                              | 28        |
| 5.7.6    | Preview-ul interactiv                                               | 28        |
| 5.7.7    | Persistare și integrare cu restul aplicației                        | 28        |
| 5.7.8    | Persistența locală a conversației AI                                | 29        |
| 5.8      | Decizii de implementare cu impact major                             | 29        |
| 5.9      | Concluzii intermediare                                              | 30        |
| <b>6</b> | <b>Generarea asistată de AI și conversia interfețelor în cod</b>    | <b>31</b> |
| 6.1      | De ce nu este suficientă generarea directă                          | 31        |
| 6.2      | Principiul pipeline-ului în trei faze                               | 31        |
| 6.3      | Faza 1: extragerea intenției                                        | 32        |
| 6.4      | Faza 2: generarea structurii                                        | 32        |
| 6.5      | Faza 3: stilizarea                                                  | 33        |
| 6.6      | Integrarea cu OpenAI API                                            | 33        |
| 6.7      | Structura cererilor AI și feedback incremental                      | 33        |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| 6.8      | Rolul ieșirilor structurate . . . . .                          | 35        |
| 6.9      | Mecanisme de validare și reparare . . . . .                    | 35        |
| 6.10     | Reprezentarea intermediară în detaliu . . . . .                | 36        |
| 6.11     | Maparea dintre canvas și IR . . . . .                          | 36        |
| 6.12     | Exemplu concret de flux de la prompt la IR și export . . . . . | 36        |
| 6.13     | Motorul de conversie . . . . .                                 | 37        |
| 6.13.1   | Separarea în proiect dedicat . . . . .                         | 37        |
| 6.13.2   | Validarea . . . . .                                            | 37        |
| 6.13.3   | Generarea single-page și multi-page . . . . .                  | 37        |
| 6.13.4   | Responsive output . . . . .                                    | 37        |
| 6.14     | Legătura dintre AI și convertor . . . . .                      | 38        |
| 6.15     | Limitări actuale . . . . .                                     | 38        |
| 6.16     | Concluzii intermediare . . . . .                               | 39        |
| <b>7</b> | <b>Securitate, testare și validare</b>                         | <b>40</b> |
| 7.1      | Rolul acestei etape în proiect . . . . .                       | 40        |
| 7.2      | Măsuri de securitate implementate . . . . .                    | 40        |
| 7.2.1    | Autentificare și sesiune . . . . .                             | 40        |
| 7.2.2    | Limitarea expunerii API-ului . . . . .                         | 40        |
| 7.2.3    | Middleware și antete de securitate . . . . .                   | 40        |
| 7.2.4    | Upload-uri și asset-uri . . . . .                              | 41        |
| 7.3      | Strategia de testare . . . . .                                 | 41        |
| 7.3.1    | Testare automată backend . . . . .                             | 41        |
| 7.3.2    | Testarea convertorului . . . . .                               | 42        |
| 7.3.3    | Scenarii reprezentative ancorate în teste . . . . .            | 42        |
| 7.3.4    | Indicatori cantitativi observați . . . . .                     | 43        |
| 7.3.5    | Testare frontend . . . . .                                     | 44        |
| 7.4      | Validare funcțională . . . . .                                 | 44        |
| 7.5      | Limitări ale validării actuale . . . . .                       | 45        |
| 7.6      | Concluzii intermediare . . . . .                               | 45        |
| <b>8</b> | <b>Concluzii și perspective de dezvoltare</b>                  | <b>46</b> |
| 8.1      | Gradul de îndeplinire a obiectivelor . . . . .                 | 46        |
| 8.2      | Puncte forte ale proiectului . . . . .                         | 46        |
| 8.3      | Limitări . . . . .                                             | 46        |
| 8.4      | Perspective de dezvoltare . . . . .                            | 47        |
| 8.4.1    | Colaborare și versionare . . . . .                             | 47        |
| 8.4.2    | Îmbunătățirea pipeline-ului AI . . . . .                       | 47        |
| 8.4.3    | Componentizare și design systems . . . . .                     | 47        |
| 8.4.4    | Testare frontend și observabilitate . . . . .                  | 47        |
| 8.5      | Încheiere . . . . .                                            | 47        |
| <b>A</b> | <b>Scenarii de demo și validare funcțională</b>                | <b>48</b> |
| A.1      | Scopul anexei . . . . .                                        | 48        |
| A.2      | Scenariul 1: creare cont și autentificare . . . . .            | 48        |
| A.3      | Scenariul 2: activare și folosire 2FA . . . . .                | 48        |
| A.4      | Scenariul 3: creare proiect și editare în canvas . . . . .     | 49        |
| A.5      | Scenariul 4: generare AI . . . . .                             | 49        |
| A.6      | Scenariul 5: export în cod . . . . .                           | 49        |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| A.7      | Scenariul 6: explorare, social și fork . . . . .           | 49        |
| <b>B</b> | <b>Exemple tehnice și note de arhitectură</b>              | <b>51</b> |
| B.1      | Maparea dintre modulele proiectului . . . . .              | 51        |
| B.2      | Exemplu simplificat de reprezentare intermediară . . . . . | 51        |
| B.3      | Exemple simplificate de cereri către API . . . . .         | 52        |
| B.4      | Exemplu simplificat de cod generat . . . . .               | 52        |
| B.5      | Note practice despre organizarea editorului . . . . .      | 53        |
| B.6      | Exemplu simplificat de evenimente de streaming . . . . .   | 53        |
| <b>C</b> | <b>Inventar de endpoint-uri API</b>                        | <b>55</b> |
| C.1      | Observații generale . . . . .                              | 55        |
| C.2      | Endpoint-uri pentru cont și autentificare . . . . .        | 55        |
| C.3      | Endpoint-uri pentru proiecte . . . . .                     | 55        |
| C.4      | Endpoint-uri pentru utilizatori și profiluri . . . . .     | 56        |
| C.5      | Endpoint-uri pentru explore și recomandări . . . . .       | 56        |
| C.6      | Endpoint-uri pentru AI și conversie . . . . .              | 57        |
|          | <b>Bibliografie</b>                                        | <b>58</b> |

## Listă de figuri

|     |                                                                                     |    |
|-----|-------------------------------------------------------------------------------------|----|
| 3.1 | Vedere simplificată asupra stack-ului tehnologic folosit de Favigon . . . . .       | 13 |
| 4.1 | Diagrama de use-case simplificată pentru funcționalitățile principale din Favigon   | 16 |
| 4.2 | Arhitectura logică generală a sistemului . . . . .                                  | 17 |
| 4.3 | Vedere de tip clean architecture pentru backend-ul Favigon . . . . .                | 18 |
| 4.4 | Model conceptual simplificat al datelor principale . . . . .                        | 19 |
| 4.5 | ERD simplificat pentru schema fizică a bazei de date . . . . .                      | 19 |
| 5.1 | Vedere structurală simplificată a principalelor clase și servicii backend . . . . . | 27 |
| 5.2 | Servicii principale implicate în funcționarea editorului canvas . . . . .           | 29 |
| 6.1 | Pipeline-ul AI în trei faze folosit în Favigon . . . . .                            | 32 |
| 6.2 | Diagrama de secvență simplificată pentru fluxul AI cu streaming . . . . .           | 35 |
| 6.3 | Fluxul design-to-code bazat pe reprezentarea intermediară . . . . .                 | 38 |
| 7.1 | Straturi complementare de validare folosite în proiect . . . . .                    | 45 |



## Listă de tabele

|     |                                                                             |    |
|-----|-----------------------------------------------------------------------------|----|
| 2.1 | Comparație între Favigon și aplicații relevante . . . . .                   | 7  |
| 3.1 | Servicii și volume definite în orchestrarea locală a proiectului . . . . .  | 12 |
| 4.1 | Trasabilitatea dintre obiectivele proiectului, module și validare . . . . . | 15 |
| 5.1 | Rute frontend principale și rolul lor în aplicație . . . . .                | 23 |
| 5.2 | Controllere API și responsabilitățile lor dominante . . . . .               | 26 |
| 6.1 | Câmpuri relevante din cererile AI folosite de backend . . . . .             | 34 |
| 7.1 | Măsurile de securitate implementate în Favigon . . . . .                    | 41 |
| 7.2 | Distribuția testelor automate backend . . . . .                             | 42 |
| 7.3 | Scenarii reprezentative verificate prin teste automate . . . . .            | 43 |
| 7.4 | Indicatori cantitativi observați în validarea curentă . . . . .             | 43 |
| A.1 | Scenariul de bază pentru cont și autentificare . . . . .                    | 48 |
| A.2 | Scenariul pentru autentificare în doi pași . . . . .                        | 48 |
| A.3 | Scenariul principal de lucru în editor . . . . .                            | 49 |
| A.4 | Scenariul de generare asistată de AI . . . . .                              | 49 |
| A.5 | Scenariul de export al proiectului . . . . .                                | 50 |
| A.6 | Scenariul de reutilizare socială . . . . .                                  | 50 |
| B.1 | Maparea dintre proiectele din soluție și responsabilitățile lor . . . . .   | 51 |
| C.1 | Endpoint-uri principale din zona de autentificare . . . . .                 | 55 |
| C.2 | Endpoint-uri principale pentru proiecte . . . . .                           | 56 |
| C.3 | Endpoint-uri principale pentru utilizatori . . . . .                        | 56 |
| C.4 | Endpoint-uri din zona explore . . . . .                                     | 57 |
| C.5 | Endpoint-uri pentru AI și converter . . . . .                               | 57 |

# Capitolul 1

## Introducere

### 1.1 Contextul actual al temei

În ultimii ani, dezvoltarea produselor digitale a devenit tot mai dependentă de colaborarea dintre roluri diferite: designeri, frontend developeri, backend developeri, product manageri și, din ce în ce mai des, sisteme automate bazate pe inteligență artificială. Dincolo de faptul că aceste roluri folosesc instrumente diferite, ele gândesc produsul și la niveluri de abstractizare diferite. Designerul lucrează cu layout, ierarhie vizuală și consistență. Dezvoltatorul web lucrează cu componente, stare, apeluri HTTP, validări și performanță. În practică, diferența dintre aceste două perspective produce aproape mereu un cost suplimentar în procesul de implementare.

Într-o aplicație web clasică de tip SPA, interfața nu mai este doar o succesiune de pagini statice. Ea este rezultatul unor stări multiple, a rutării client-side și a interacțiunilor asincrone cu serverul, ceea ce face ca proiectarea și implementarea să fie strâns legate [14, 1]. În același timp, ecosistemul modern a produs o gamă largă de instrumente specializate: unele pun accent pe design și handoff, altele pe no-code, altele pe generare de cod sau pe conținut. Deși fiecare rezolvă o parte a problemei, fluxul complet rămâne adesea fragmentat.

Aplicații precum Figma, Webflow, Framer sau Builder.io ilustrează foarte bine această tendință. Figma oferă un mediu matur pentru design și inspecție tehnică, cu suport pentru Dev Mode și conectare la cod [15]. Webflow mută accentul spre construcția vizuală a site-urilor și publicarea rapidă [30]. Framer împinge zona de prototipare și generare de layout cu ajutorul AI [16]. Builder.io propune un model de dezvoltare vizuală cu integrare în cod și AI orientat spre reutilizarea componentelor existente [8, 9]. Tocmai această diversitate arată că problema nu este încă închisă și că există loc pentru experimente și soluții integrate.

### 1.2 Problema pe care o urmărește lucrarea

Problema de la care pornește lucrarea este una concretă: cum poate fi redusă distanța dintre ideea de interfață, reprezentarea ei vizuală și codul care ajunge efectiv să ruleze într-un browser? În fluxul obișnuit, utilizatorul descrie o nevoie, designerul desenează o soluție, dezvoltatorul o reconstruiește în cod, iar echipa verifică manual dacă rezultatul este apropiat de intenția inițială. Fiecare transfer dintre etape introduce timp de așteptare, ambiguitate și posibile erori.

Din această observație rezultă trei subprobleme. Prima este problema editării vizuale: utilizatorul are nevoie de un spațiu în care să poată construi o pagină, să repoziționeze elemente, să lucreze cu mai multe pagini și să își poată salva progresul fără operații tehnice inutile. A doua este problema structurării interfeței: dacă se dorește folosirea AI sau exportul de cod, nu este suficientă existența unei imagini convingătoare, ci este necesară o reprezentare internă coerentă, stabilă și validabilă. A treia este problema reutilizării: odată ce un design există, el ar trebui să poată fi publicat, apreciat, partajat și eventual preluat de alți utilizatori.

În multe produse existente aceste trei subprobleme sunt tratate separat. Există editoare vizuale foarte bune, dar fără export tehnic serios. Există generatoare AI interesante, dar greu de controlat. Există convertoare către cod, dar care pornesc de la intrări rigide și nu oferă un spațiu de lucru propriu-zis. În cadrul acestui proiect, ele sunt tratate împreună, într-o platformă unitară.

Formulat direct, ideea proiectului este următoarea: Favigon este o platformă web pentru designeri, frontend developeri, studenți și echipe mici care doresc să reducă distanța dintre concept, prototip editabil și cod executabil. Accentul nu cade pe publicarea imediată a unui site, ci pe construirea unui flux coerent în care interfața poate fi desenată, structurată, validată, exportată și apoi reutilizată.

### 1.3 Motivația alegerii temei

Tema a fost aleasă deoarece reunește mai multe direcții relevante pentru un proiect de licență. În primul rând, are o componentă clară de arhitectură software. Un produs de tipul Favigon nu poate fi realizat doar ca interfață sau doar ca API, deoarece valoarea lui apare tocmai din cooperarea mai multor subsisteme. Din acest motiv, tema a impus decizii reale de separare a responsabilităților, persistare, modelare a datelor și organizare a codului.

În al doilea rând, tema are o componentă puternică de experiență a utilizatorului. Editorul canvas nu este util dacă doar “funcționează” tehnic. El trebuie să fie suficient de fluid încât utilizatorul să simtă că poate experimenta fără fricțiune mare. Asta înseamnă zoom, selecție, pan, istoric, clipboard, editare text, previzualizare și salvare automată. Practic, multe decizii de implementare au fost influențate direct de comportamentul așteptat în interfață.

În al treilea rând, integrarea AI într-un produs real reprezintă un subiect actual și util, dar numai dacă este tratată pragmatic. Interesul nu este folosirea inteligenței artificiale ca element decorativ, ci ca mecanism care poate accelera generarea unei structuri inițiale și poate reduce timpul dintre idee și rezultat. De aceea este evitată o soluție bazată doar pe prompt și este introdus un pipeline etapizat, cu validare și constrângeri explicite prin JSON Schema [31, 17, 24].

### 1.4 Obiectivele proiectului

Obiectivul general al proiectului este proiectarea și implementarea unei platforme web care să permită:

- crearea și editarea vizuală a interfețelor web într-un canvas;
- generarea asistată de AI a unei structuri inițiale pentru pagini;
- conversia interfețelor construite într-o reprezentare intermediară coerentă;
- exportul automat al rezultatului în HTML, React și Angular;
- publicarea și reutilizarea proiectelor într-un spațiu cu funcții sociale.

Pentru a atinge acest obiectiv general, au fost urmărite și o serie de obiective specifice:

1. proiectarea unei arhitecturi full-stack suficient de clare încât logica de business, persistența și integrarea externă să nu fie amestecate;
2. definirea unui model de date care să acopere utilizatorii, proiectele, relațiile sociale și legăturile externe de autentificare;
3. dezvoltarea unui editor canvas care să poată gestiona mai multe pagini și mai multe tipuri de operații de editare;
4. introducerea unei reprezentări intermediare reutilizabile atât de editor, cât și de motorul de conversie;
5. dezvoltarea unui pipeline AI în mai multe faze, pentru a crește controlul asupra ieșirii;
6. validarea soluției atât prin testare automată, cât și prin scenarii funcționale.

Pe parcursul implementării, a fost păstrată o regulă simplă: fiecare funcționalitate nouă trebuie să aducă valoare clară în produs și să poată fi justificată tehnic. Din acest motiv, anumite

idei care ar fi mărit spectaculos aplicația, dar fără un câștig real de coerență, au fost lăsate pentru dezvoltări viitoare.

## 1.5 Contribuțiile proprii

Aportul personal din această lucrare se vede în principal în șase direcții.

Prima direcție este integrarea într-un produs unitar a unor funcții care, de regulă, apar în produse diferite: editor vizual, generare AI, conversie în cod și distribuire socială. Miza nu a fost doar adăugarea mai multor module, ci și funcționarea lor pe aceeași bază de date, pe aceeași logică de autentificare și pe același model intern.

A doua direcție este organizarea editorului canvas. În locul unei implementări monolitice, logica este separată pe servicii focalizate pentru viewport, istoric, clipboard, persistare, gesturi și randare. Această decizie nu este vizibilă imediat la nivel de interfață, dar a contat mult pentru stabilitatea dezvoltării ulterioare.

A treia direcție este reprezentarea intermediară a interfeței. Editorul nu lucrează doar cu elemente vizuale izolate, iar exportul nu pornește direct de la DOM. Între aceste niveluri există un model intern bazat pe noduri, layout, stil și poziționare, care face posibilă atât validarea, cât și exportul în mai multe tehnologii.

A patra direcție este pipeline-ul AI în trei faze: intenție, structură și stil. Această separare este utilă deoarece generarea directă a codului sau a designului final este mai greu de controlat. În schimb, dacă problema este spartă în pași mai mici, fiecare pas poate fi verificat mai ușor și, la nevoie, reparat.

A cincea direcție este motorul de conversie. Acesta nu generează doar un singur rezultat static, ci suportă export single-page, multi-page și responsive. În plus, aceeași bază este folosită pentru HTML, React și Angular, ceea ce obligă la o abstracție mai curată decât în cazul unui export dedicat unei singure ținte.

A șasea direcție este componenta de produs: autentificare cu 2FA, OAuth, proiecte publice și private, explore, like, star, follow și fork. Aceste funcții schimbă proiectul dintr-un simplu editor tehnic într-o platformă care poate susține colaborare și reutilizare.

## 1.6 Metoda de lucru

Din punct de vedere practic, dezvoltarea aplicației a fost iterativă. Procesul a început cu funcționalitățile de bază necesare pentru existența produsului: autentificare, proiecte și editor minimal. După aceea, au fost adăugate treptat comportamente mai complexe în editor, persistare automată și suport pentru export. Abia după ce această bază a devenit suficient de stabilă a fost integrat pipeline-ul AI, deoarece fără un model intern coerent rezultatele generate ar fi fost greu de controlat.

Pe partea de backend a fost urmărită o structură apropiată de principiile prezentate în literatura de arhitectură software [5, 19]. Asta a însemnat separarea API-ului de logica de aplicație, a logicii de aplicație de infrastructură și folosirea interfețelor între straturi. Nu a fost urmărită o implementare dogmatică, ci una practică: suficient de clară ca să poată susține extinderea proiectului, dar fără un volum de abstracții inutile.

Pe partea de frontend, strategia a fost păstrarea logicii specifice editorului în interiorul feature-ului canvas, nu într-o zonă generică de utilitare sau componente partajate. Din moment ce editorul concentrează cea mai mare parte din complexitatea aplicației, această decizie a ajutat la evitarea dispersării comportamentelor și la păstrarea contextului tehnic aproape de ecranele care îl folosesc.

## 1.7 Structura lucrării

Lucrarea este organizată în opt capitole principale și trei anexe.

Capitolul al doilea analizează domeniul și compară Favigon cu aplicații existente. Capitolul al treilea prezintă fundamentele teoretice și tehnologiile folosite. Capitolul al patrulea tratează analiza cerințelor și proiectarea sistemului. Capitolul al cincilea descrie implementarea platformei la nivel de module și fluxuri principale. Capitolul al șaselea este dedicat generării asistate de AI și conversiei din design în cod, acesta fiind și nucleul tehnic al lucrării. Capitolul al șaptelea abordează securitatea, testarea și validarea. Ultimul capitol sintetizează concluziile și direcțiile de dezvoltare.

Anexele completează partea principală cu scenarii demo, note de arhitectură și o vedere de ansamblu asupra API-ului. Această structură separă mai clar fundamentarea teoretică de implementarea efectivă și păstrează contribuțiile personale vizibile în partea centrală a lucrării, nu ascunse între noțiuni generale.

## Capitolul 2

# Analiza domeniului și studiu comparativ

## 2.1 Evoluția spațiului design-to-code

Zona de *design-to-code* nu este nouă, dar a devenit mult mai vizibilă odată cu maturizarea aplicațiilor web și cu apariția modelelor generative moderne. În literatura mai veche, problema era formulată deseori ca transformarea unei descrieri vizuale într-un cod de interfață. Un exemplu cunoscut este *pix2code*, unde autorul tratează generarea codului din capturi de ecran ale interfeței [6]. Mai recent, lucrările dedicate modelelor mari de limbaj și modelelor multimodale arată că interesul s-a mutat de la simpla reconstrucție a aspectului spre păstrarea structurii, a layout-ului și a intenției de produs [28, 32].

În practică, piața a evoluat într-o direcție puțin diferită de lucrările academice. Produsele comerciale au observat că utilizatorul nu are nevoie doar de un model care produce cod, ci de un întreg flux de lucru: proiectare, organizare pe pagini, colaborare, verificare, export, publicare și iterație. Din acest motiv, instrumentele moderne nu mai pot fi analizate doar prin întrebarea “generează sau nu cod?”, ci prin ansamblul de funcții pe care îl oferă.

Mai există și un detaliu important. În multe situații, problema nu este că nu se poate genera ceva, ci că rezultatul generat nu este suficient de ușor de integrat într-un produs real. Un layout poate arăta bine ca demonstrație, dar dacă nu are o structură logică, dacă nu respectă constrângeri tehnice și dacă nu poate fi controlat prin iterații mici, valoarea lui practică scade. Această observație a influențat direct proiectarea Favigon: în loc să trateze AI-ul ca pe un generator final, platforma îl include într-un lanț controlat, în care reprezentarea intermediară și validarea joacă un rol central.

## 2.2 Criterii de comparație

Pentru a poziționa corect proiectul, au fost alese criterii relevante atât din punct de vedere tehnic, cât și din punct de vedere al utilității pentru utilizator:

- **editare vizuală directă:** existența unui spațiu de lucru de tip canvas sau editor vizual;
- **asistență AI:** posibilitatea de a porni de la un prompt, sugestii sau generare automată de structură;
- **design-to-code:** existența unei conversii către cod sau măcar a unor artefacte pentru handoff tehnic;
- **orientare spre produs final:** suport pentru publicare, organizare pe pagini sau dezvoltare continuă;
- **partajare și colaborare:** mecanisme prin care rezultatele pot fi distribuite sau reutilizate;
- **control tehnic al ieșirii:** cât de mult poate fi influențată structura rezultatului și cât de ușor poate fi integrat în cod existent.

Aceste criterii nu sunt perfecte și nu încearcă să stabilească “cel mai bun” produs. Scopul lor este mai modest: să arate unde se află Favigon pe o hartă a instrumentelor existente și ce gol încearcă să acopere.

## 2.3 Aplicații relevante din aceeași zonă

### 2.3.1 Figma

Figma este, în mod firesc, un punct de referință pentru orice discuție despre proiectarea interfețelor. Din perspectiva acestei lucrări, interesul nu este dat doar de partea de design colaborativ, ci și de Dev Mode. Documentația oficială arată clar că Dev Mode este gândit ca spațiu pentru inspectarea designurilor și pentru apropierea dintre design și implementare [15]. Cu toate acestea, Figma rămâne în primul rând un instrument de design și handoff. Chiar dacă poate genera fragmente de cod sau poate conecta design system-uri la bazele de cod, el nu devine automat un mediu complet de construire și publicare a produsului.

Pentru Favigon, Figma este util mai ales ca termen de comparație în două direcții. Prima direcție este ideea de structură vizuală clară și multi-page. A doua este faptul că handoff-ul către dezvoltare nu este suficient în sine; dacă se urmăresc iterații rapide, este necesar un spațiu în care designul să poată fi nu doar inspectat, ci și transformat mai departe în ceva executabil.

### 2.3.2 Webflow

Webflow este relevant deoarece reprezintă foarte bine zona de *visual website builder*. Pagina oficială pune accent pe construcția vizuală, publicare și generare rapidă de site-uri [30]. Avantajul evident al acestei abordări este că rezultatul ajunge repede online și este ușor de gestionat din punct de vedere al conținutului. În schimb, Webflow are un focus clar pe zona de website publishing și CMS, nu pe un model general de interfață reutilizabil între mai multe framework-uri.

Pentru poziționarea Favigon, Webflow este important tocmai pentru a delimita intenția aplicației. Nu a fost urmărită construirea încă unui publicator vizual de site-uri. Accentul cade mai mult pe controlul tehnic al structurii, exportul în cod și legătura dintre un editor vizual și o reprezentare internă care poate fi convertită.

### 2.3.3 Framer

Framer este interesant fiindcă mută discuția mai aproape de zona AI. Pagina oficială pentru Wireframer arată o abordare în care un prompt este folosit pentru a construi rapid un layout responsive, cu accent pe structură și flux [16]. Este o direcție apropiată de cea urmărită și aici, însă diferența majoră este că în Favigon este făcută explicită separarea dintre intenție, structură și stilizare.

Cu alte cuvinte, Framer demonstrează că generarea ghidată de prompt poate fi utilă pentru pornirea rapidă a unui proiect. Totuși, pentru o lucrare inginerescă este mai valoros să fie explicați și controlați pașii interni ai generării, nu doar să fie observat rezultatul final.

### 2.3.4 Builder.io

Builder.io este probabil cel mai apropiat reper comercial pentru direcția urmărită în această lucrare. Platforma lor pune accent pe dezvoltare vizuală, AI și reutilizarea componentelor existente [8, 9]. Ideea de a nu porni de la zero de fiecare dată, ci de a avea o punte între design, componente și cod, este una foarte relevantă.

Diferența principală este că Builder.io este orientat puternic spre integrarea într-un ecosistem existent și spre productivitate în contexte enterprise. Favigon, în schimb, este un proiect academic și experimental, în care accentul este pus pe claritatea modelului intern și pe trasabilitatea tehnică a fiecărei etape. Asta înseamnă că produsul nu concurează direct cu Builder.io la nivel de maturitate comercială, dar tratează o parte din probleme într-un mod mai transparent pentru analiză și explicație.

## 2.4 Tabel comparativ

**Tabela 2.1:** Comparație între Favigon și aplicații relevante

| Criteriu                                          | Figma                    | Webflow                   | Framer  | Builder.io            | Favigon |
|---------------------------------------------------|--------------------------|---------------------------|---------|-----------------------|---------|
| Editor vizual direct                              | Da                       | Da                        | Da      | Da                    | Da      |
| Pornire din prompt AI                             | Parțial                  | Da                        | Da      | Da                    | Da      |
| Model intern orientat spre export multi-framework | Nu explicit              | Nu                        | Limitat | Parțial               | Da      |
| Export către HTML/React/Angular                   | Nu ca obiectiv principal | Nu în forma urmărită aici | Limitat | Orientat pe integrare | Da      |
| Funcții sociale în aceeași platformă              | Nu                       | Nu                        | Nu      | Nu                    | Da      |
| Publicare proiecte proprii                        | Parțial                  | Da                        | Da      | Da                    | Parțial |
| Reutilizare prin fork / star / like               | Nu                       | Nu                        | Nu      | Nu                    | Da      |
| Accent pe controlul etapelor AI                   | Nu                       | Nu                        | Parțial | Parțial               | Da      |

Tabelul 2.1 nu are rolul de a demonstra că Favigon este “mai bun” decât produsele consacrate. Ar fi o comparație nerealistă. Scopul lui este să arate că Favigon combină, într-o formă academică și experimentală, elemente care în alte produse apar separate: editor vizual, asistență AI, export în cod și funcții sociale.

## 2.5 Poziționarea proiectului Favigon

În urma comparației, Favigon poate fi poziționat ca o platformă experimentală de tip *AI-assisted visual builder*, orientată mai puțin spre publicare imediată și mai mult spre explorarea unui flux coerent între idee, design, model intern și cod executabil. Cu alte cuvinte, produsul nu își propune să concureze prin maturitatea ecosistemului, ci prin claritatea traseului tehnic pe care îl oferă.

În termeni practici, produsul este mai potrivit pentru prototipare rapidă, experimente de interfață, proiecte academice și aplicații mici sau medii decât pentru scenarii enterprise foarte rigide. Această delimitare este importantă, deoarece explică mai bine valoarea reală a platformei: nu promite înlocuirea unui ecosistem comercial matur, ci oferă un cadru integrat pentru a trece mai repede de la idee la un rezultat executabil și analizabil tehnic.

Această poziționare vine și cu limite. De exemplu, Favigon nu are nivelul de rafinament vizual sau de integrare enterprise al unor produse comerciale mature. Nici partea de colaborare simultană în timp real nu este încă implementată. Totuși, pentru scopul lucrării, aceste limite nu sunt un dezavantaj major. Ele permit concentrarea pe întrebările urmărite aici:



- cum pot reprezenta o interfață într-o formă suficient de stabilă încât să o editez și să o export;
- cum pot integra AI într-un mod controlabil;
- cum pot susține reutilizarea prin funcții sociale simple;
- cum pot organiza întregul produs astfel încât să rămână clar tehnic.

## 2.6 Golul pe care îl acoperă Favigon

Din analiza de mai sus rezultă că există un spațiu intermediar între instrumentele clasice de design, generatoarele AI rapide și platformele orientate strict spre publicare. Favigon încearcă să acopere tocmai acest spațiu intermediar. Mai exact, platforma propune:

1. un editor propriu, nu doar o zonă de import sau inspectare;
2. un model intermediar explicit, nu doar un rezultat final opac;
3. export multi-framework, nu doar un cod specific unei singure ținte;
4. funcții sociale integrate, astfel încât proiectele să poată circula în comunitate;
5. o integrare a AI-ului care poate fi analizată, validată și îmbunătățită pas cu pas.

Din punct de vedere academic, acesta reprezintă un unghi bun pentru o lucrare de licență. Proiectul nu se reduce la folosirea unui model AI pentru obținerea unor pagini, ci pune în discuție mai multe aspecte ingineresti: modelare internă, arhitectură pe straturi, validare, testare, UX și decizii de produs.

## 2.7 Concluzii intermediare

Analiza domeniului arată că proiectul Favigon nu apare într-un vid, ci într-un context în care instrumentele pentru design, dezvoltare vizuală și AI cresc rapid. În același timp, ea arată și de ce proiectul are sens ca temă de licență. Există deja instrumente puternice, dar nu toate tratează împreună editarea vizuală, generarea controlată, exportul tehnic și partajarea în aceeași platformă.

Concluzia acestui capitol este că Favigon nu trebuie evaluat ca un produs comercial finalizat, ci ca o platformă software care încearcă să demonstreze fezabilitatea unui flux integrat. Pe baza acestei concluzii, capitolul următor trece de la piață și comparații la fundamentele teoretice și tehnologiile pe care s-a sprijinit implementarea.

## Capitolul 3

# Fundamente teoretice și tehnologii utilizate

### 3.1 Considerații generale

Înainte de a descrie implementarea concretă, este utilă clarificarea fundamentelor pe care se sprijină proiectul. Favigon nu este doar o colecție de ecrane și endpoint-uri, ci o combinație între mai multe idei: aplicație single-page, API REST, autentificare pe bază de token, persistare relațională, orchestrare containerizată, generare de conținut structurată și conversie dintr-un model intermediar către cod executabil.

Aceste noțiuni sunt tratate împreună deoarece în proiect ele nu apar izolat. De exemplu, editorul canvas este implementat pe frontend, dar depinde direct de modul în care proiectele sunt persistate în backend. Pipeline-ul AI produce o structură intermediară, dar valoarea ei reală se vede abia când motorul de conversie o transformă în fișiere HTML, React sau Angular. Din acest motiv, capitolul de față urmărește să explice nu doar *ce* tehnologii sunt folosite, ci și *de ce* alegerea lor a fost potrivită pentru problema rezolvată.

### 3.2 Aplicații single-page și rolul frontend-ului modern

Favigon folosește pe partea de client o aplicație Angular 20 organizată sub forma unui SPA. Într-un SPA, browserul încarcă inițial o singură pagină, iar apoi navigarea între ecrane este gestionată în principal client-side, fără reîncărcarea completă a documentului [1]. Acest model este potrivit pentru Favigon din două motive.

Primul motiv este experiența utilizatorului. Editorul canvas presupune interacțiuni dese, actualizări vizuale rapide și trecere fluidă între zone precum profil, explore, settings, proiect și preview. Dacă fiecare navigare ar depinde de reîncărcarea integrală a paginii, aplicația ar deveni mai greu de folosit și mai puțin apropiată de instrumentele moderne de design.

Al doilea motiv este organizarea tehnică. Angular oferă rutare, componente, injecție de dependențe, gestiune a formularelor și integrare cu HTTP într-un ecosistem unitar. În plus, proiectul folosește componente standalone și 'provideRouter', ceea ce simplifică structura aplicației și reduce dependența de module tradiționale [2, 4]. Pentru un proiect în care cea mai mare complexitate se află în interfață, această coerență a framework-ului a contat mult.

Un element important în implementare este folosirea *signals*. Documentația Angular prezintă signals ca mecanism reactiv granular pentru urmărirea și actualizarea stării [3]. În editorul canvas, această abordare este utilă deoarece permite reacții mai fine la schimbarea selecției, a viewport-ului, a paginii curente sau a istoricului, fără a introduce un strat extern de management al stării. Pentru un feature foarte interactiv, această soluție oferă un raport bun între simplitate și control.

### 3.3 Arhitectura client-server și API-ul REST

Comunicarea dintre frontend și backend este organizată sub forma unui API REST. În literatura clasică, stilul REST este tratat ca o modalitate de proiectare a sistemelor software bazate pe resurse, interacțiuni standardizate și separarea clară între client și server [14]. În Favigon, această alegere a fost una pragmatică.

Aplicația are resurse destul de clar delimitate: utilizatori, proiecte, design-uri, upload-uri, rezultate generate, relații sociale și operații de autentificare. Pentru fiecare dintre acestea a fost naturală definirea unor controllere și endpoint-uri specializate. Spre exemplu, proiectele sunt gestionate prin endpoint-uri dedicate pentru creare, actualizare, design, thumbnail, assets, like, star sau fork, în timp ce autentificarea este separată în propriul controller.

Am ales ASP.NET Core 9 pentru backend deoarece oferă un cadru foarte potrivit pentru API-uri moderne: rutare declarativă, middleware configurabil, DI integrat, autentificare, autorizare și extensibilitate la nivel de pipeline HTTP. Dincolo de faptul că framework-ul este matur, el permite și o separare curată între straturile aplicației. În proiect, API-ul doar primește cereri și returnează răspunsuri, în timp ce logica de business este ținută în servicii de aplicație, iar persistența și integrarea externă sunt mutate în infrastructură.

### 3.4 Autentificare, token-uri și securitate la nivel de cerere

Zona de autentificare din Favigon combină mai multe mecanisme: login și register clasic, OAuth cu furnizori externi, refresh token și autentificare în doi pași. Din perspectiva acestei lucrări, important este modul în care ele sunt integrate coerent.

ASP.NET Core oferă suport direct pentru autentificare JWT bearer [7]. În proiect, token-ul de acces este transportat prin cookie HTTP only, iar backend-ul îl extrage din cookie-ul 'jwt' în cadrul configurării 'JwtBearer'. Practic, acest lucru combină avantajul token-ului auto-conținut cu avantajul unui transport mai greu de accesat din JavaScript. Această alegere nu elimină orice risc posibil, dar este rezonabilă pentru tipul de aplicație construit aici.

În plus, proiectul folosește refresh token separat și limitează calea cookie-ului de refresh la endpoint-ul de reînnoire, ceea ce reduce suprafața de expunere. Pentru operații sensibile sau scenarii de risc mai mare, este introdusă și autentificare în doi pași pe bază de cod temporar transmis prin email. Din perspectiva ghidurilor OWASP, folosirea unui factor suplimentar este una dintre cele mai eficiente măsuri pentru reducerea atacurilor care pornesc de la compromiterea parolei [25, 26].

Dincolo de autentificare, backend-ul folosește și rate limiting. Documentația ASP.NET Core tratează rate limiting-ul ca mecanism de control al fluxului de cereri, util pentru protecție împotriva abuzului și pentru stabilitate [18]. În proiect sunt definite politici diferite pentru zone precum autentificare, AI, converter și users, tocmai pentru că profilul de risc și costul operațiilor nu sunt aceleași.

### 3.5 Reprezentarea intermediară a interfeței

Una dintre ideile de bază ale proiectului este că designul nu trebuie legat direct nici de canvas, nici de HTML final. Între aceste două niveluri este introdusă o reprezentare intermediară bazată pe noduri ('IRNode'), layout, stil, poziționare, metadata și variante responsive. Din punct de vedere conceptual, această reprezentare joacă rolul unui limbaj intern al aplicației.

Avantajul principal al unui astfel de model este separarea preocupărilor. Editorul poate să manipuleze elemente și structuri fără să producă imediat cod pentru browser. În același timp, motorul de conversie poate genera HTML, React sau Angular fără să depindă de detaliile de implementare ale editorului. Asta înseamnă că aceeași structură poate fi folosită în două scopuri diferite: editare și export.

În plus, modelul intermediar permite validare înainte de generare. Dacă un arbore de noduri este incomplet, inconsistent sau imposibil de randat corect, problema poate fi detectată înainte de etapa de export. Acesta este un avantaj important față de abordările care merg direct

din prompt în cod. În Favigon, tocmai această validare intermediară face posibilă combinarea AI-ului cu un flux tehnic controlabil.

### 3.6 JSON Schema și ieșiri structurate pentru AI

În momentul în care s-a decis folosirea unui model AI pentru generarea interfețelor, una dintre primele probleme a fost fiabilitatea formatului de ieșire. Un răspuns foarte bun ca limbaj natural nu este automat util pentru o aplicație software. Este nevoie de date structurate, de câmpuri verificabile și de o formă care să poată fi procesată programatic.

Pentru această nevoie, JSON Schema este un instrument foarte potrivit. Specificația definește un mod standardizat de a exprima structura, tipurile și restricțiile unui document JSON [31, 17]. În Favigon, schema este folosită pentru a constrânge ieșirile modelului AI atunci când se generează atât blueprint-ul intenției, cât și arborele intermediar al interfeței.

Din perspectiva OpenAI API, această idee este compatibilă cu *structured outputs*, adică solicitarea unui răspuns care respectă o anumită schemă [24, 23]. Totuși, faptul că formatul este corect nu garantează automat că și conținutul este potrivit. De aceea, în implementare nu s-a mers doar până la schema JSON, ci au fost adăugate și validări semantice, plus un mecanism de reparare atunci când modelul produce ieșiri invalide sau incomplete.

### 3.7 Baza de date relațională și persistența

Pentru persistență este folosit PostgreSQL, prin Entity Framework Core și providerul Npgsql [27, 22]. Alegerea unei baze de date relaționale a fost firească, deoarece datele principale au relații clare: un utilizator poate avea mai multe proiecte, proiectele pot avea aprecieri și bookmark-uri, utilizatorii se pot urmări între ei, iar un proiect poate fi fork-uit din alt proiect.

Pe lângă modelul relațional clasic, aplicația păstrează și un câmp de tip JSON serializat pentru design-ul proiectului. Aici există un compromis asumat. Pe de o parte, datele vizuale ale canvas-ului sunt prea flexibile pentru a fi modelate comod în tabele relaționale fine. Pe de altă parte, utilizatorii, relațiile sociale și metadatele proiectelor se potrivesc foarte bine într-un model relațional. În practică, combinația dintre tabele relaționale și document JSON pentru design s-a dovedit suficient de echilibrată pentru scopul proiectului.

În implementarea concretă, schema rezultată este compactă, dar suficient de expresivă pentru produsul construit. Baza de date conține tabelele `users`, `projects`, `linked_accounts`, `user_follows`, `project_bookmarks` și `project_likes`. Tabelele relaționale descriu identitatea utilizatorului, proiectele, relațiile sociale și interacțiunile dintre utilizatori și proiecte, iar câmpul `DesignJson` din `projects` păstrează documentul de canvas sub formă `jsonb`. Această alegere permite filtrare și agregare eficientă pentru metadate, fără a forța fragmentarea structurii vizuale în numeroase tabele auxiliare.

Schema fizică include și constrângeri relevante pentru consistență. Email-ul utilizatorului este unic, conturile OAuth au unicitate atât pe perechea `(Provider, ProviderUserId)`, cât și pe perechea `(UserId, Provider)`, iar tabelele de tip `follow`, `bookmark` și `like` folosesc chei compuse pentru a preveni dublurile. În plus, proiectele folosesc o autoreferință prin `'ForkedFromProjectId'`, ceea ce face posibilă trasabilitatea unui fork fără a duplica separat noțiunea de proiect-sursă. Contextul EF Core completează automat timestamp-urile `'CreatedAt'` și `'UpdatedAt'`, astfel încât persistența să rămână coerentă și la nivel operațional, nu doar la nivel de structură.

## 3.8 Orchestrare și infrastructură

La nivel de infrastructură, proiectul folosește Docker Compose pentru definirea și pornirea serviciilor principale: baza de date PostgreSQL, API-ul, frontend-ul și tunelul Cloudflare [13, 12]. Această variantă reduce efortul necesar pentru pornirea locală și descrie într-un singur fișier dependențele dintre servicii, volumele persistente și rețeaua internă.

Avantajul principal al acestui mod de lucru este repetabilitatea. Când toate componentele importante sunt definite declarativ, mediul devine mai ușor de refăcut, de demonstrat și de mutat între sisteme. Pentru o lucrare de licență, acest aspect nu este deloc minor, deoarece ușurează demo-ul și reduce riscul ca aplicația să funcționeze doar pe o singură configurație locală.

În configurația actuală, fișierul `docker-compose.yml` nu pornește doar containerele, ci exprimă și câteva decizii operaționale utile. Baza de date are *healthcheck*, astfel încât API-ul să nu pornească înainte ca PostgreSQL să fie pregătit pentru conexiuni. API-ul montează separat volumul pentru upload-uri, ceea ce permite persistența imaginilor și thumbnail-urilor independent de ciclul containerului. Frontend-ul este construit cu argument explicit pentru fișierul de mediu de producție, iar tunelul Cloudflare este atașat peste aplicația deja pornită, nu folosit ca substitut pentru rețeaua internă.

**Tabela 3.1:** Servicii și volume definite în orchestrarea locală a proiectului

| Serviciu / volum | Rol principal                                         | Observații utile                                                                    |
|------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------|
| db               | instanță PostgreSQL 16 pentru persistența relațională | folosește <i>healthcheck</i> cu <code>pg_isready</code> și volum persistent dedicat |
| api              | aplicația ASP.NET Core expusă intern pe portul 8080   | depinde de sănătatea bazei de date și montează directorul de upload-uri             |
| frontend         | aplicația Angular compilată pentru mediu de producție | depinde de API și rulează în aceeași rețea internă                                  |
| cloudflared      | publicare externă controlată prin Cloudflare Tunnel   | folosește token extern și pornește doar după frontend                               |
| postgres_data    | persistă datele PostgreSQL                            | evită pierderea datelor la recrearea containerelor                                  |
| uploads_data     | persistă fișierele încărcate și thumbnail-urile       | separă conținutul generat de ciclul de build/deploy                                 |

Pentru expunerea aplicației în exterior este folosit și Cloudflare Tunnel. Documentația oficială subliniază faptul că tunelul funcționează prin conexiuni doar outbound dintre infrastructura locală și rețeaua Cloudflare, evitând expunerea directă a unui IP public și reducând suprafața de atac [11]. În proiect, această alegere a fost utilă mai ales pentru publicare și testare externă, fără a complica prea mult configurarea rețelei. Vederea simplificată a stack-ului rezultat este redată în Figura 3.1.



## Capitolul 4

# Analiza și proiectarea sistemului

### 4.1 Pornirea de la cerințe

Înainte de implementarea funcționalităților principale, a fost necesară separarea cât mai clară a cerințelor sistemului. Pentru acest tip de proiect, riscul cel mai mare este adăugarea multor funcții atractive, dar fără o structură clară. De aceea, analiza a pornit de la întrebarea simplă: ce trebuie să poată face utilizatorul și ce trebuie să poată susține sistemul în spate?

Din perspectiva utilizatorului final, aplicația trebuie să permită crearea unui cont, autentificarea, administrarea profilului, lucrul cu proiecte și folosirea editorului vizual. Din perspectiva fluxului de produs, trebuie să existe și operații precum publicarea unui proiect, explorarea proiectelor altora, aprecierea lor și reutilizarea prin fork. Din perspectiva obiectivului tehnic al lucrării, sistemul trebuie să poată genera designuri cu ajutorul AI și să exporte rezultatul în cod.

Aceste cerințe au fost apoi grupate în două categorii mari: funcționale și nefuncționale.

### 4.2 Cerințe funcționale

Principalele cerințe funcționale identificate pentru Favigon sunt următoarele:

1. utilizatorul poate crea cont, autentifica și ieși din aplicație;
2. utilizatorul poate folosi login clasic sau autentificare prin furnizori externi;
3. utilizatorul poate activa sau dezactiva autentificarea în doi pași;
4. utilizatorul își poate edita profilul și imaginea de profil;
5. utilizatorul poate crea, edita și șterge proiecte;
6. pentru fiecare proiect, utilizatorul poate salva designul și thumbnail-ul asociat;
7. utilizatorul poate edita vizual proiectul într-un canvas cu mai multe pagini;
8. utilizatorul poate genera o structură de design prin AI, inclusiv în regim streaming;
9. utilizatorul poate valida și exporta proiectul în HTML, React sau Angular;
10. utilizatorul poate marca proiectele altora cu like sau star;
11. utilizatorul poate urmări alți utilizatori și poate vedea proiecte recomandate;
12. utilizatorul poate fork-ui proiecte publice și le poate adapta în propriul spațiu.

La nivel de implementare, aceste cerințe se reflectă destul de direct în controllerele și serviciile backend, respectiv în feature-urile frontend. De exemplu, 'AccountController' și 'AuthService' acoperă fluxurile de autentificare, 'ProjectsController' și 'ProjectService' gestionează ciclul de viață al proiectelor, iar 'AiDesignController' împreună cu serviciile AI acoperă generarea asistată. Pentru a păstra legătura dintre obiective, module și validare, Tabelul 4.1 sintetizează corespondențele principale.

### 4.3 Cerințe nefuncționale

Pe lângă cerințele funcționale, proiectul are și câteva cerințe nefuncționale importante:

- **claritate arhitecturală:** logica de business nu trebuie amestecată în controllere;
- **extensibilitate:** exportul către mai multe framework-uri trebuie să fie posibil fără rescrierea editorului;

- **performanță interactivă:** editorul trebuie să rămână fluid pentru operațiile uzuale;
- **siguranță:** autentificarea și operațiile sensibile trebuie protejate rezonabil;
- **persistență robustă:** designul proiectelor trebuie salvat automat și recuperat coerent;
- **portabilitate:** aplicația trebuie să poată fi pornită ușor local și demonstrată într-un mediu apropiat de producție;
- **testabilitate:** cel puțin partea de backend și motorul de conversie trebuie să poată fi verificate prin teste automate.

Aceste cerințe nefuncționale au influențat puternic structura aplicației. Necesitatea de a exporta în mai multe tehnologii a dus aproape inevitabil la introducerea reprezentării intermediare. Necesitatea de a păstra editorul fluid a dus la împărțirea logicii în servicii specializate pe frontend. Necesitatea de a limita amestecul de responsabilități a dus la structurarea backend-ului pe straturi.

**Tabela 4.1:** Trasabilitatea dintre obiectivele proiectului, module și validare

| Obiectiv                                | Module principale                                                             | Formă de validare                                                          |
|-----------------------------------------|-------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Autentificare și administrare cont      | 'AccountController', 'AuthService', 'UsersController'                         | teste backend pentru autentificare și scenarii funcționale de login / 2FA  |
| Editor vizual multi-page                | 'CanvasPage', 'CanvasEditor-StateService', 'CanvasPageManagerService'         | scenarii de editare, autosave și restaurare de stare                       |
| Reprezentare intermediară reutilizabilă | mapper-e canvas $\leftrightarrow$ IR, 'IR-Node', 'ConverterService'           | validare structurală înainte de export și teste pentru convertor           |
| Generare asistată de AI controlabilă    | 'AiDesignController', 'AiDesignService', 'AiPipelineService', 'OpenAiClient'  | validare structurală, mecanism de auto-reparare și scenarii funcționale AI |
| Export multi-framework                  | 'ConverterController', 'ConverterService', 'ConverterEngine'                  | teste pentru HTML / React / Angular, responsive și multi-page              |
| Componentă socială și reutilizare       | 'ProjectsController', 'UserController', 'ExploreController', 'ProjectService' | teste service/controller și scenarii de publicare, like, star și fork      |

## 4.4 Actorii principali

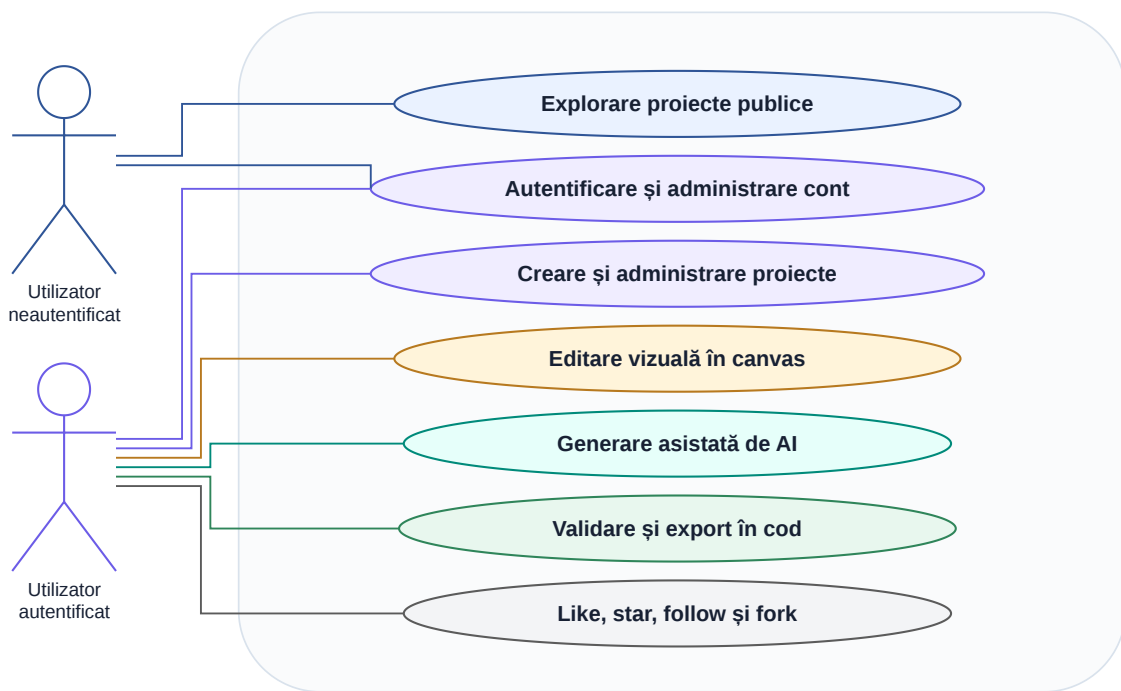
Sistemul are două categorii principale de actori.

**Utilizatorul neautentificat** poate accesa paginile publice, poate vedea proiectele expuse și poate explora conținutul public. El nu poate modifica proiecte și nu poate folosi editorul complet.

**Utilizatorul autentificat** reprezintă actorul central al sistemului. Acesta își poate administra profilul, își poate crea și edita proiectele, poate folosi editorul canvas, poate apela generarea AI, poate exporta cod și poate interacționa social cu alți utilizatori.

Pentru a sintetiza relația dintre actori și funcționalitățile dominante, Figura 4.1 prezintă o vedere de tip use-case. Ea nu urmărește toate endpoint-urile sau toate stările posibile, ci capacitățile care structurează produsul la nivel de utilizare.





**Figura 4.1:** Diagrama de use-case simplificată pentru funcționalitățile principale din Favigon

## 4.5 Scenarii principale de utilizare

### 4.5.1 Scenariul 1: creare și editare proiect

Utilizatorul se autentifică, creează un proiect nou și ajunge în editorul canvas. Acolo poate adăuga elemente, poate crea pagini noi, poate modifica proprietăți și poate salva implicit progresul. La ieșire sau la schimbarea paginii, sistemul trebuie să se asigure că starea relevantă este persistată.

### 4.5.2 Scenariul 2: generare asistată de AI

Utilizatorul pornește de la un prompt și solicită generarea unei interfețe. Backend-ul trimite cererea către serviciul AI, primește etapele pipeline-ului și returnează rezultatul. Frontend-ul transformă structura obținută în elemente de canvas și permite utilizatorului să continue editarea manuală. Acest scenariu este important deoarece arată că AI-ul este integrat în fluxul editorului, nu separat de el.

### 4.5.3 Scenariul 3: export în cod

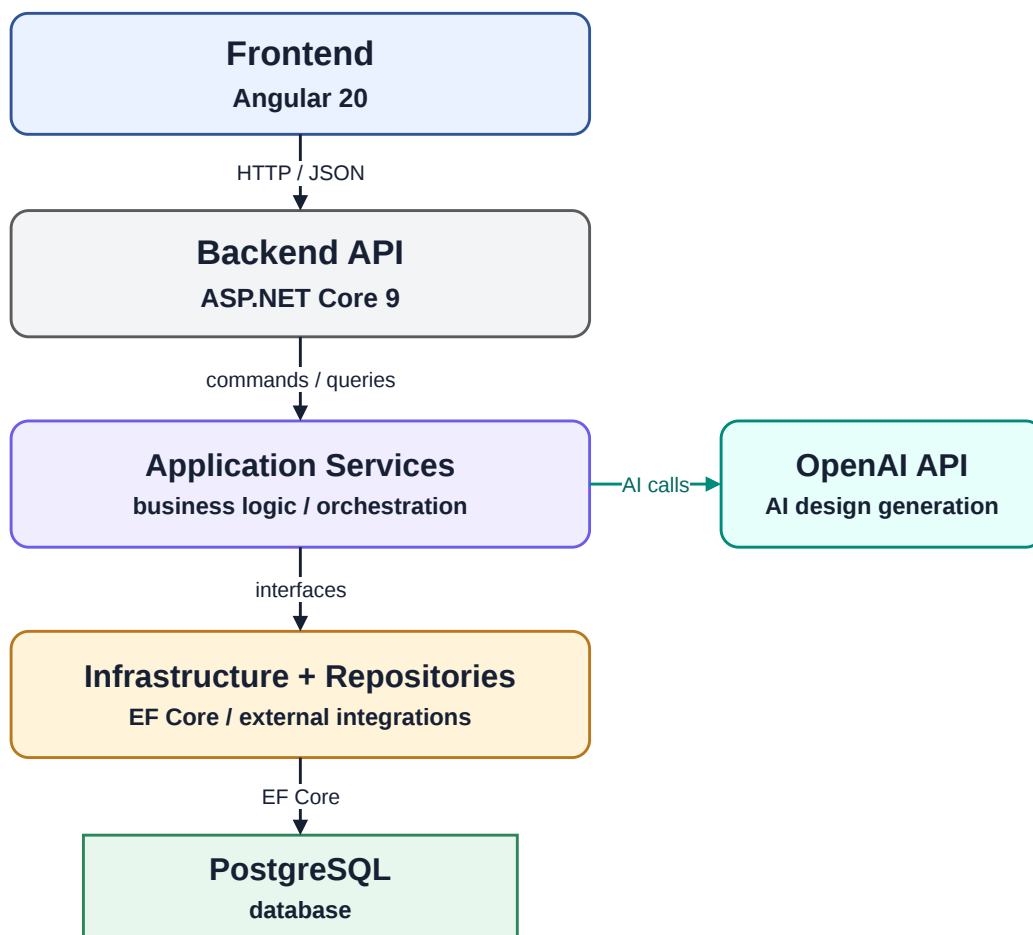
După ce proiectul ajunge într-o stare satisfăcătoare, utilizatorul cere validarea și exportul. Designul este convertit în reprezentarea intermediară, verificat și transmis motorului de conversie. Rezultatul este un set de fișiere pentru framework-ul ales. Acest scenariu justifică existența convertorului ca modul separat.

### 4.5.4 Scenariul 4: distribuire și reutilizare

Un utilizator publică un proiect, iar alt utilizator îl descoperă în explore sau pe profilul autorului. Poate să îl aprecieze, să îl salveze sau să îl fork-uiască. În acest fel, proiectul capătă și o dimensiune de comunitate, nu rămâne doar un document privat.

## 4.6 Arhitectura generală

La nivel înalt, sistemul este împărțit în frontend, backend, bază de date și servicii externe. Frontend-ul Angular gestionează interfața, rutarea și starea locală a editorului. Backend-ul ASP.NET Core expune endpoint-uri REST și coordonează logica de aplicație. PostgreSQL stochează datele persistente. Serviciile externe includ OpenAI API pentru generare și Cloudflare Tunnel pentru expunere controlată. Arhitectura logică rezultată este redată în Figura 4.2.



**Figura 4.2:** Arhitectura logică generală a sistemului

Această împărțire nu este doar una de prezentare. Ea se regăsește concret în structura soluției backend: proiectele 'Favigon.API', 'Favigon.Application', 'Favigon.Infrastructure', 'Favigon.Domain', 'Favigon.Converter' și 'Favigon.Tests'. Fiecare are un rol clar, iar motorul de conversie este separat explicit deoarece are propriile reguli și propria logică de transformare.

## 4.7 Organizarea backend-ului pe straturi

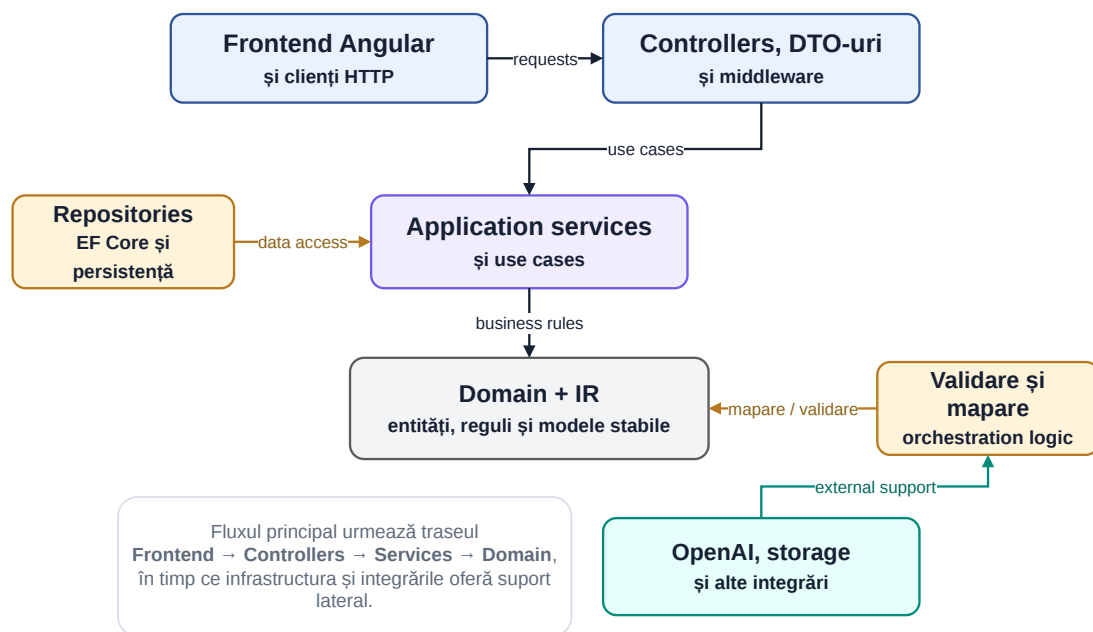
La nivel de backend, fluxul tipic al unei cereri este:

1. controller-ul primește cererea HTTP și verifică aspectele de intrare;
2. serviciul de aplicație aplică logica de business;
3. repository-ul sau adaptorul extern execută persistența sau integrarea necesară;
4. rezultatul este mapat și întors către client.

Această structură reduce riscul ca un controller să devină prea încărcat. De exemplu, un endpoint din 'ProjectsController' nu decide singur cum se construiește un slug, cum se șterg asset-urile sau cum se tratează un fork. Toate acestea sunt delegate către 'ProjectService', care la rândul lui colaborează cu repository-ul și cu storage-ul pentru assets.

Avantajul acestei structuri este vizibil și în testare. Pentru că serviciile sunt relativ bine delimitate, ele pot fi verificate independent de transportul HTTP. De aceea, în proiect există teste directe pentru 'AuthService', 'ProjectService', 'UserService' și 'ConverterEngine', nu doar teste la nivel de controller.

Aceeași organizare poate fi interpretată și printr-o lentilă apropiată de *clean architecture*. Regulile esențiale ale produsului și reprezentările interne trebuie să rămână în centru, iar infrastructura, transportul HTTP și integrarea cu servicii externe trebuie să depindă de acest nucleu, nu invers. Figura 4.3 sintetizează această idee pentru backend-ul Favigon.



**Figura 4.3:** Vedere de tip clean architecture pentru backend-ul Favigon

## 4.8 Organizarea frontend-ului pe feature-uri

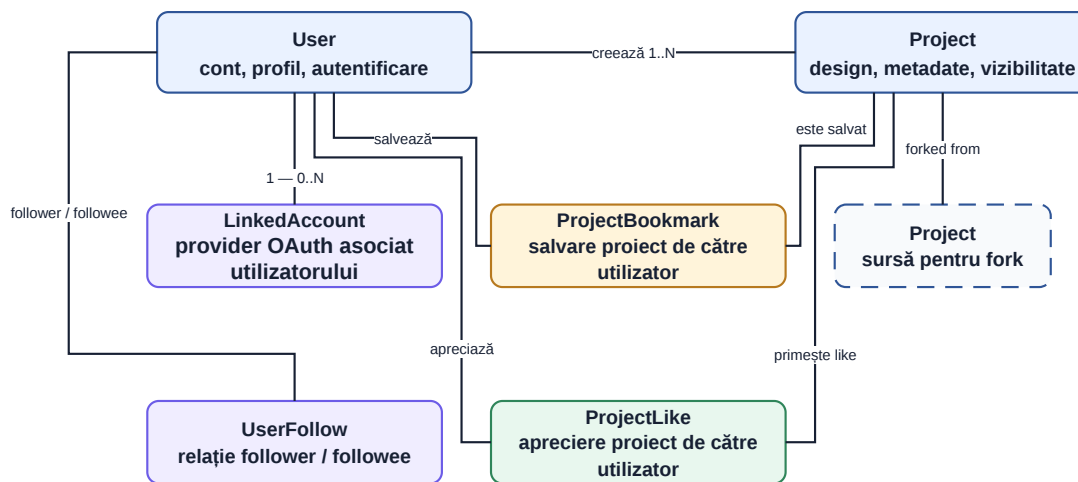
Frontend-ul este organizat în jurul conceptelor 'core', 'shared' și 'features'. 'core' conține modelele, serviciile API, interceptorii și utilitarele globale. 'shared' conține componente reutilizabile. 'features' grupează zonele funcționale ale aplicației: autentificare, canvas, explore, profil, settings și stars.

Această structură este potrivită pentru proiect deoarece cea mai mare complexitate este concentrată într-un singur feature, anume 'canvas'. În loc să împrăștii logica editorului în toată aplicația, serviciile, mapper-ele și utilitarele specifice au fost păstrate aproape de acest feature. În practică, această alegere a redus numărul de dependențe surpriză și a făcut fluxul de lucru mai ușor de urmărit.

## 4.9 Modelul de date conceptual

Din punct de vedere conceptual, modelul de date are câteva entități centrale: utilizatorul, proiectul, conturile externe legate de utilizator, relațiile de follow dintre utilizatori și relațiile de like sau bookmark dintre utilizatori și proiecte. Designul efectiv al unui proiect este stocat ca

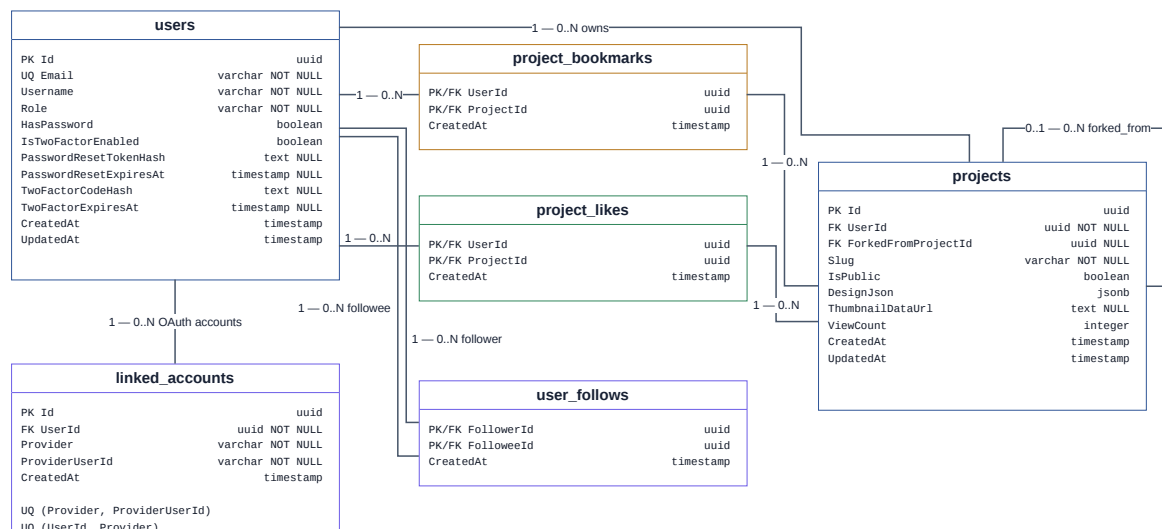
JSON în cadrul proiectului, deoarece structura lui este prea flexibilă pentru o modelare exclusiv relațională. Vederea conceptuală simplificată apare în Figura 4.4.



**Figura 4.4:** Model conceptual simplificat al datelor principale

Din cod se observă și câteva decizii importante. Proiectul are câmpuri pentru ‘Slug’, ‘IsPublic’, ‘ThumbnailDataUrl’, ‘ViewCount’ și ‘ForkedFromProjectId’. Utilizatorul are câmpuri pentru ‘HasPassword’, ‘IsTwoFactorEnabled’, datele de resetare a parolei și date temporare pentru fluxul 2FA. Aceste câmpuri arată că modelul este orientat clar spre un produs complet, nu doar spre stocarea unui desen.

Pentru a trece de la nivelul de domeniu la nivelul de implementare, Figura 4.5 prezintă un ERD simplificat extras din configurația Entity Framework Core. El evidențiază entitățile persistente, cheile principale și relațiile care merită discutate la nivel de licență.



**Figura 4.5:** ERD simplificat pentru schema fizică a bazei de date

ERD-ul scoate în evidență și câteva constrângeri importante din implementare: unicitatea email-ului pentru utilizatori, unicitatea conturilor externe pe perechile (Provider, ProviderUserId) și (UserId, Provider), cheile compuse pentru relațiile sociale și auto-referința ForkedFromProjectId din projects. Astfel, schema nu stochează doar resurse izolate, ci și legăturile necesare pentru reutilizare, explorare și trasabilitatea fork-urilor.

Această structură arată explicit și compromisul de modelare adoptat în proiect. Datele cu structură stabilă, cum sunt identitatea utilizatorului, relațiile sociale și metadatele proiectelor, rămân relaționale. În schimb, documentul efectiv al interfeței este păstrat compact în DesignJson, deoarece arborele de elemente, stiluri și layout-uri este prea flexibil pentru a fi mapat eficient în tabele relaționale fine. În plus, timestamp-urile CreatedAt și UpdatedAt sunt menținute automat în contextul de persistență, ceea ce reduce riscul de inconsistență între creare, actualizare și export ulterior.

## 4.10 Fluxurile principale

### 4.10.1 Fluxul de autentificare

Fluxul de autentificare începe cu un request către 'AccountController', este procesat în 'AuthService', iar la final backend-ul emite cookie-urile necesare. Dacă utilizatorul are 2FA activ, autentificarea este împărțită în două etape: validarea credențialelor și validarea codului temporar. Această separare este reflectată și în cod, prin token-ul temporar specific fluxului 2FA.

### 4.10.2 Fluxul de editare

La deschiderea unui proiect, frontend-ul încarcă datele proiectului și apoi designul asociat. 'CanvasPersistenceService' coordonează persistarea automată și sincronizarea thumbnail-ului, iar starea editorului este împărțită între mai multe servicii specializate. Din punct de vedere al utilizatorului, totul ar trebui să pară unitar. Din punct de vedere intern, însă, fluxul este descompus în pași mai mici tocmai pentru a păstra controlul asupra lui.

### 4.10.3 Fluxul de export

Pentru export, editorul produce reprezentarea intermediară, apoi o trimite către backend prin 'ConverterController'. 'ConverterService' validează structura și apelează 'ConverterEngine', care generează fișierele pentru framework-ul ales. Acest flux este important deoarece justifică separarea convertorului într-un proiect distinct.

### 4.10.4 Fluxul AI

Pentru generare, frontend-ul trimite un prompt către 'AiDesignController'. Backend-ul poate răspunde fie clasic, fie prin streaming SSE. În spate, 'AiPipelineService' parcurge fazele de intenție, structură și stil, iar rezultatul final este transformat în date pe care frontend-ul le poate integra în editor.

## 4.11 Decizii de proiectare care au influențat cel mai mult implementarea

Privind retrospectiv, există câteva decizii de proiectare care au avut impact major asupra întregului proiect.

Prima decizie a fost separarea clară între canvas și codul final exportat. O abordare bazată pe editarea directă a HTML-ului sau a DOM-ului ar fi produs probabil un sistem mai fragil și mai greu de extins.

A doua decizie a fost tratarea AI-ului ca pe un subsistem controlabil, nu ca pe o cutie neagră. De aici rezultă atât folosirea JSON Schema, cât și împărțirea pipeline-ului.

A treia decizie a fost păstrarea logicii editorului în servicii specializate. Fără acest pas, componenta principală de canvas ar fi devenit foarte repede imposibil de întreținut.

A patra decizie a fost folosirea unei baze relaționale pentru metadate și a unui câmp JSON pentru design. Deși este un compromis, el s-a dovedit eficient pentru tipul de informații gestionate.

## **4.12 Concluzii intermediare**

În urma analizei și proiectării, sistemul rezultat are o formă coerentă atât la nivel de produs, cât și la nivel tehnic. Cerințele au fost traduse într-o arhitectură cu straturi clare, actori bine definiți, fluxuri importante și un model de date adaptat problemei.

Capitolul următor trece de la această vedere de ansamblu la implementarea concretă a platformei: autentificare, proiecte, funcții sociale și editorul canvas.

## Capitolul 5

# Implementarea platformei Favigon

### 5.1 Privire de ansamblu asupra implementării

După etapa de proiectare, implementarea efectivă a platformei a urmărit aceeași idee centrală: fiecare zonă importantă a produsului trebuie să aibă responsabilități clare și puncte de integrare ușor de urmărit. În practică, asta a însemnat că nu a fost dezvoltat întâi un editor izolat și abia apoi restul produsului, ci aplicația a evoluat ca întreg.

Mai exact, partea de autentificare, management de proiecte și persistare a fost dezvoltată relativ devreme, pentru că fără ele editorul ar fi rămas un demo local. Apoi au fost consolidate editorul canvas și suportul pentru export. Într-o etapă ulterioară au fost integrate fluxurile AI și a fost extinsă componenta socială, astfel încât proiectele să poată fi nu doar create, ci și publicate, descoperite și reutilizate.

Rezultatul este o aplicație în care modulele sunt diferite ca rol, dar strâns legate între ele. De exemplu, editorul depinde de proiecte, proiectele depind de autentificare, generarea AI depinde de editor și de modelul intermediar, iar funcțiile sociale depind de modelul relațional al utilizatorilor și proiectelor.

### 5.2 Rutare și pornirea aplicației frontend

Pe partea de client, aplicația este configurată prin `ApplicationConfig`, folosind `provideRouter`, `provideHttpClient` cu interceptori și `provideAnimations`. Această alegere este în linie cu stilul modern Angular bazat pe componente standalone. Din punct de vedere practic, ea reduce necesitatea unor module centrale voluminoase și face mai clară relația dintre rutare, infrastructura HTTP și comportamentul global al aplicației.

Un detaliu util este activarea `provideZoneChangeDetection` cu `eventCoalescing`. Pentru un produs cu editor vizual, acest tip de optimizare este relevant deoarece reduce numărul de actualizări declanșate de evenimente succesive și ajută la păstrarea unei interfețe mai fluide în scenarii de drag, selecție sau input repetat.

Rutarea este organizată astfel încât paginile principale să fie încărcate lazy și protejate diferențiat. Ruta rădăcină nu trimite utilizatorul într-o pagină statică, ci încearcă să încarce utilizatorul curent și redirecționează dinamic fie spre profilul acestuia, fie spre `/login`. În plus, editorul canvas folosește un `canDeactivate guard` care forțează flush-ul persistenței înainte de părăsirea paginii, ceea ce leagă direct infrastructura de navigare de cerința de robustețe a autosave-ului.

**Tabela 5.1:** Rute frontend principale și rolul lor în aplicație

| Rută                   | Scop                                   | Observații de acces / navigare                                                                   |
|------------------------|----------------------------------------|--------------------------------------------------------------------------------------------------|
| /login                 | autentificare și intrare în sistem     | protejată de <code>loginEntryGuard</code> , pentru a evita revenirea inutilă după login          |
| /reset-password        | recuperare acces și setare parolă nouă | rută publică, separată de fluxul clasic de autentificare                                         |
| /project/:slug         | editorul canvas principal              | protejată cu <code>authGuard</code> și cu <code>canDeactivate</code> pentru flush de persistență |
| /project/:slug/preview | preview HTML al proiectului            | necesită autentificare și reutilizează datele de proiect pentru randare separată                 |
| /explore               | descoperire de proiecte și utilizatori | accesibilă ca punct de intrare în componenta socială                                             |
| /settings              | configurări de profil și cont          | protejată pentru utilizatorul autentificat                                                       |
| /stars                 | proiecte salvate de utilizator         | afișează bookmark-urile proprii                                                                  |
| /:username             | profilul utilizatorului curent         | folosit și ca destinație implicită după încărcarea sesiunii                                      |

Interacțiunea dintre rutare și infrastructura HTTP este completată de doi interceptori importanți: unul pentru trimiterea credențialelor către API și altul pentru încercarea automată de refresh atunci când sesiunea curentă expiră. Din punct de vedere al utilizatorului, această logică face ca aplicația să pară mai continuă și mai coerentă; din punct de vedere tehnic, ea mută comportamentele repetitive într-un strat transversal, în loc să fie duplicate în fiecare pagină sau serviciu.

## 5.3 Modulul de autentificare și profil

### 5.3.1 Fluxurile de bază

Modulul de autentificare este implementat în jurul ‘AccountController’ și al serviciului ‘AuthService’. La nivel de API există endpoint-uri pentru register, login, logout, refresh, resetare parolă, setare parolă, schimbare parolă, autentificare prin GitHub și Google, precum și pentru activarea sau dezactivarea autentificării în doi pași.

Pe partea de backend, ‘AuthService’ face mai mult decât o simplă verificare de credențiale. El normalizează datele de intrare, verifică unicitatea utilizatorului, gestionează hash-uirea parolei, tratează fluxurile OAuth, construiește token-urile JWT și emite un token temporar pentru etapa intermediară a autentificării în doi pași. În plus, aceeași zonă gestionează și resetarea parolei prin token transmis prin email.

Din punct de vedere al produsului, autentificarea nu este tratată ca o singură operație de tip “email + parolă”. În practică, utilizatorii pot intra în sistem prin mai multe căi și pot avea conturi legate la furnizori diferiți. Din acest motiv există și entitatea ‘LinkedAccount’, folosită pentru asocierea unui utilizator local cu identități externe.



### 5.3.2 JWT în cookie și refresh token

O decizie importantă a fost transportul token-ului de acces prin cookie HTTP only. În implementare, controller-ul nu întoarce token-ul către frontend pentru stocare manuală în 'localStorage', ci îl plasează în cookie-ul 'jwt'. Refresh token-ul este păstrat separat, într-un cookie dedicat cu path limitat la endpoint-ul de refresh.

Această alegere simplifică partea de client și reduce expunerea unor date sensibile în codul din browser. În același timp, introduce și obligația de a trata atent expirațiile și reînnoirea sesiunii. De aceea, în frontend există și interceptorul de refresh, care încearcă revalidarea sesiunii când întâlnește un răspuns care indică expirarea token-ului de acces.

### 5.3.3 Autentificarea în doi pași

În cadrul proiectului, 2FA este implementat prin cod temporar trimis prin email. La prima vedere, acesta nu este cel mai avansat model posibil, dar pentru scopul aplicației este o soluție rezonabilă și ușor de demonstrat. În plus, implementarea este structurată destul de curat: există flux separat pentru activare, dezactivare și verificare la login, iar token-ul temporar pentru sesiunea de autentificare incompletă este diferit de token-ul normal de acces.

Această funcționalitate nu rămâne la nivel de extensie izolată. Ea este integrată atât în serviciul de autentificare, cât și în controller, în opțiuni și în fluxul de UI, ceea ce îi conferă rolul unei măsuri reale de securitate operațională.

### 5.3.4 Profilul utilizatorului

Pe lângă autentificare, utilizatorul își poate administra profilul: nume afișat, bio, imagine și legături OAuth. 'UserController' expune endpoint-uri pentru profilul curent, încărcarea imaginii de profil, căutarea de utilizatori și vizualizarea profilurilor publice. În plus, există endpoint-uri pentru follow și unfollow, precum și pentru listele de followers și following.

Această parte a aplicației are un rol dublu. Pe de o parte este o funcționalitate de produs normală. Pe de altă parte, ea oferă baza pentru zona socială și pentru ideea că proiectele au autori, pot fi urmărite și pot circula între utilizatori.

## 5.4 Gestionarea proiectelor

### 5.4.1 Ciclul de viață al proiectelor

'ProjectService' este una dintre cele mai importante piese din backend, deoarece concentrează logica legată de proiecte. El gestionează crearea proiectului, actualizarea metadatelor, ștergerea, salvarea designului, salvarea thumbnail-ului, upload-ul de imagini, bookmark-urile, like-urile și operația de fork.

La creare, proiectul primește un slug unic derivat din nume. La modificare, dacă numele se schimbă, slug-ul este recalculat. La ștergere, serviciul se ocupă și de curățarea asset-urilor asociate. Acest detaliu este important deoarece evită acumularea de fișiere orfane și arată că proiectul nu tratează persistența doar la nivel de bază de date, ci și la nivelul fișierelor încărcate.

### 5.4.2 Persistarea designului

Pentru design-ul propriu-zis al unui proiect, backend-ul păstrează JSON-ul serializat în câmpul 'DesignJson' al entității 'Project'. Pe frontend, 'CanvasPersistenceService' coordonează încărcarea, transformarea și salvarea acestui document.

Un aspect important este că salvarea nu este exclusiv manuală. Editorul folosește persistare automată cu debounce și mecanisme de flush atunci când utilizatorul părăsește pagina sau

proiectul trebuie sincronizat imediat. Acest comportament este important pentru UX. Dacă utilizatorul lucrează într-un editor de design și trebuie să apese explicit *Save* după fiecare pas important, experiența devine artificială și fragilă.

În implementare, persistarea automată este tratată explicit prin parametri observabili în cod: schimbările relevante programează o salvare după 500 ms, flush-ul așteaptă cel mult 4 s finalizarea persistenței, iar la erori temporare se folosesc până la 4 reîncercări cu backoff exponențial pornind de la 2 s. Serviciul gestionează separat stările de *isSaving*, persistarea la ieșirea din pagină și generarea de thumbnail-uri după perioade scurte de inactivitate. Aceste detalii transformă editorul dintr-un demo interactiv într-un subsistem care poate susține utilizare repetată.

### 5.4.3 Thumbnail și asset-uri

Fiecare proiect poate avea și thumbnail. În editor, thumbnail-ul este generat automat pornind din canvas, inclusiv printr-o variantă bazată pe 'html2canvas'. Pe backend, fișierul este salvat separat, iar la update se invalidează vechiul 'ThumbnailDataUrl' dacă este cazul.

În plus, proiectul suportă upload de imagini pentru asset-uri folosite în design. Validarea fișierelor se face în backend, iar stocarea este abstractizată printr-un serviciu dedicat. Această decizie a fost utilă pentru că a evitat legarea prea strânsă a logicii de proiecte de detaliile concrete ale fișierelor.

Un detaliu important, mai puțin vizibil în interfață, este ciclul de viață al asset-urilor. La resalvarea designului, backend-ul compară asset-urile prezente în documentul nou cu cele din versiunea anterioară și poate elimina resursele care au devenit orfane. În mod similar, la ștergerea unui proiect sau a unui cont, sunt curățate atât asset-urile proiectelor, cât și imaginea de profil a utilizatorului. Această zonă nu schimbă experiența vizuală în mod direct, dar este importantă pentru a arăta că aplicația tratează responsabil și consistența stocării de fișiere.

## 5.5 Funcționalități sociale și explorare

Componenta socială este implementată prin controllerele 'ExploreController', 'ProjectsController' și 'UsersController', împreună cu serviciile și repository-urile aferente. Utilizatorii pot vizualiza proiecte populare sau recomandate, pot descoperi alți utilizatori, pot aprecia proiecte, le pot salva și le pot fork-ui.

Deși această zonă nu este nucleul tehnic al lucrării, ea schimbă semnificativ caracterul produsului. Fără ea, Favigon ar fi rămas un editor personal. Cu ea, proiectul capătă logică de comunitate: există autori, conținut public, indicatori de interes și relații între utilizatori.

În implementare, 'ExploreService' tratează separat trei fluxuri: proiecte *trending*, proiecte recomandate și utilizatori sugerați. Pentru proiectele populare este folosit un set limitat de rezultate, potrivit pentru afișarea rapidă a conținutului public. Pentru recomandări, backend-ul verifică mai întâi dacă utilizatorul autentificat urmărește deja alte conturi; dacă da, încearcă să construiască un răspuns personalizat, iar dacă nu există bază suficientă sau utilizatorul este anonim, revine la o selecție generală de proiecte recente și relevante. Pentru sugestiile de persoane, serviciul returnează informații derivate precum numărul de urmăritori, numărul de proiecte publice și starea *isFollowedByCurrentUser*. Această logică oferă zonei 'explore' un comportament mai apropiat de un produs real decât de o simplă listă statică.

Operația de fork este deosebit de interesantă deoarece leagă partea socială de partea de producție. Când un utilizator face fork la un proiect public, sistemul creează un nou proiect privat pentru el, cu design-ul copiat și cu legătura către proiectul-sursă. Astfel, reutilizarea nu este doar conceptuală, ci concretă și trasabilă.

## 5.6 Suprafața API și distribuția responsabilităților

Din perspectiva backend-ului, aplicația expune o suprafață REST suficient de largă încât să merite descrisă separat. În forma actuală, există șase controllere principale: `AccountController`, `ProjectsController`, `UsersController`, `ExploreController`, `AiDesignController` și `ConverterController`. Împreună, acestea acoperă autentificarea, datele de profil, proiectele, funcțiile sociale, generarea asistată de AI și exportul în cod.

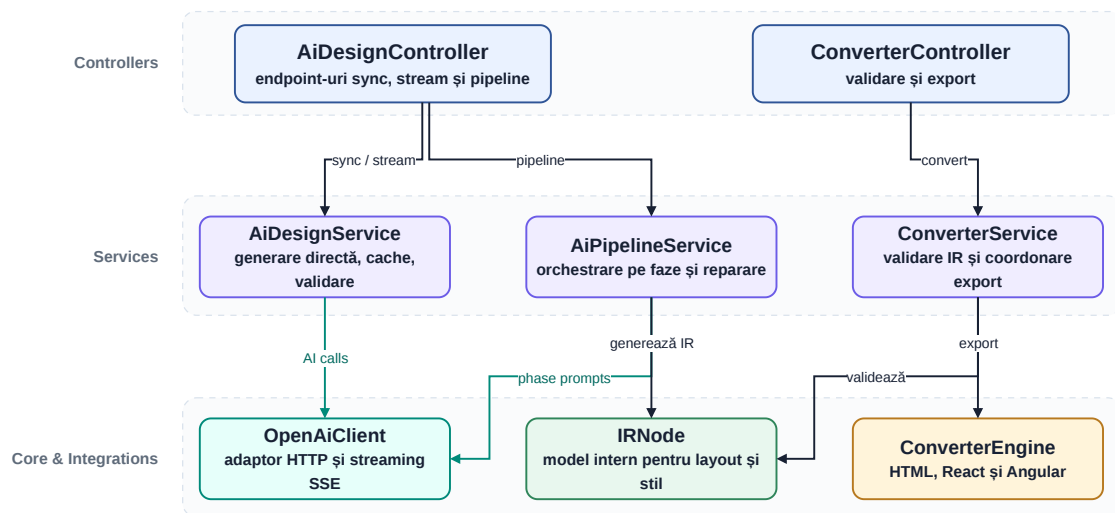
Distribuția responsabilităților pe controllere are un rol important în claritatea implementării. Zonele costisitoare sau sensibile, precum AI-ul și convertorul, sunt separate de ciclul de viață al proiectelor. În același timp, controllerele de produs, cum sunt cele pentru proiecte și utilizatori, pot rămâne focalizate pe resursele pe care le gestionează. Astfel, API-ul nu devine o colecție arbitrară de endpoint-uri, ci o hartă destul de coerentă a capabilităților aplicației.

**Tabela 5.2:** Controllere API și responsabilitățile lor dominante

| Controller                       | Resurse / operații principale                                                  | Observații utile                                                 |
|----------------------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------|
| <code>AccountController</code>   | register, login, logout, refresh, resetare parolă, OAuth, 2FA                  | concentrează logica de sesiune și intrarea în sistem             |
| <code>ProjectsController</code>  | CRUD proiecte, design, thumbnail, asset-uri, like, star, fork, view            | este controllerul cel mai bogat funcțional din zona de produs    |
| <code>UsersController</code>     | profil curent, profil public, follow/unfollow, linked accounts, imagine profil | leagă identitatea utilizatorului de componenta socială           |
| <code>ExploreController</code>   | trending, recomandări și sugestii de utilizatori                               | pune în valoare date agregate și filtrate, nu doar resurse brute |
| <code>AiDesignController</code>  | generare AI clasică, streaming, pipeline pe faze                               | include atât răspuns sincron, cât și SSE incremental             |
| <code>ConverterController</code> | validare IR, generare HTML/React/Angular, fișiere multi-page                   | separă clar validarea de generarea finală                        |

În implementare se observă și diferențierea infrastructurală a acestor zone. Controllerul AI este protejat atât prin autorizare, cât și prin politica dedicată de rate limiting ai. Controllerul de conversie are propria politică converter, iar `ProjectsController` aplică și limite diferite de dimensiune pentru upload de imagini, salvare design, flush și thumbnail. Aceste detalii arată că separarea controllerelelor nu este doar conceptuală, ci are și efecte practice asupra securității, costului și fiabilității API-ului.

Dacă diagrama de arhitectură din capitolul precedent arată straturile mari, Figura 5.1 coboară un nivel și sintetizează colaborarea dintre clasele și serviciile dominante din fluxurile AI și export. Nu este o diagramă exhaustivă a întregului backend, ci o vedere structurală orientată pe subsistemele cele mai relevante pentru contribuția tehnică a lucrării.



**Figura 5.1:** Vedere structurală simplificată a principalelor clase și servicii backend

## 5.7 Editorul canvas

### 5.7.1 Rolul editorului în întregul produs

Editorul canvas este zona cea mai complexă din frontend și, în mod realist, centrul aplicației. În jurul lui gravitează generarea AI, persistarea, exportul și o mare parte din interacțiunile vizuale ale utilizatorului. În implementare, pagina principală a editorului ('CanvasPage') este o componentă mare, dar ea funcționează mai mult ca punct de orchestrare decât ca loc în care se află toată logica.

Practic, componenta injectează o suită întreagă de servicii: stare editor, viewport, istoric, clipboard, gesturi, context menu, persistare, generare, manager de pagini, stilizare DOM și altele. Separarea urmărește principiile interacțiunii de tip *direct manipulation*, unde utilizatorul operează pe obiecte vizibile, primește feedback imediat și poate reveni rapid asupra acțiunilor [29]. În același timp, păstrarea unui model intern separat de suprafața vizuală apropie editorul de direcțiile discutate în literatura despre combinarea editării directe cu reprezentări programabile [10]. Principalele servicii implicate sunt sintetizate în Figura 5.2.

### 5.7.2 Starea editorului

'CanvasEditorStateService' păstrează semnalele principale ale editorului: paginile, pagina curentă, elementele selectate, unealta curentă și alte stări derivate. Faptul că starea este separată într-un serviciu dedicat face posibilă partajarea ei între mai multe comportamente fără a transmite manual date prin multe nivele de componente.

Pe lângă starea explicită, editorul folosește și multe valori derivate: elementul curent, limitele selecției, lista de elemente vizibile, hărți auxiliare pentru copii, indexări rapide și stări temporare de drag sau resize. În cod se vede clar că aceste structuri sunt tratate ca parte din modelul intern al editorului, nu ca simple variabile locale.

### 5.7.3 Viewport, selecție și gesturi

Pentru ca editorul să fie util, utilizatorul trebuie să poată naviga în canvas, să selecteze elemente și să le manipuleze natural. 'CanvasViewportService' controlează zoom-ul, offset-ul și proprietățile CSS asociate. 'CanvasGestureService' gestionează drag, resize, rotate, editarea de text și calculul limitelor pentru overlay-uri.

O problemă practică interesantă a fost sincronizarea dintre modelul intern și limitele reale din DOM, mai ales pentru elemente transformate sau pentru copii din layout-uri flexibile. De aceea, în implementare există noțiuni precum *stable selection bounds*, cache pentru limite și mecanisme speciale pentru cazurile în care măsurarea live ar produce efecte vizuale nedorite. Este un exemplu de complexitate care nu apare în descrierea de nivel înalt, dar devine foarte concretă într-un editor bazat pe gesturi, selecție și feedback imediat.

#### 5.7.4 Istoric și clipboard

‘CanvasHistoryService’ și ‘CanvasClipboardService’ susțin operații de tip undo, redo, copy și paste. Din punct de vedere al utilizatorului, acestea sunt funcții de bază. Din punct de vedere intern, ele sunt importante pentru că obligă sistemul să poată captura și reaplica stări coerente ale documentului.

Pentru istoric, soluția folosită este bazată pe snapshot-uri ale stării relevante. Nu este cea mai sofisticată variantă posibilă, dar pentru proiectul de față oferă un raport bun între simplitate și control. În plus, istoricul este suficient de clar încât să poată fi restaurat și după reîncărcarea proiectului.

#### 5.7.5 Managementul paginilor

Editorul nu se limitează la o singură pagină. ‘CanvasPageManagerService’ și serviciile asociate permit existența mai multor pagini în același proiect, schimbarea paginii active și organizarea elementelor pe pagini. Această decizie influențează și exportul, deoarece face posibilă generarea multi-page în backend.

Din perspectiva lucrării, suportul multi-page este important deoarece mută Favigon din zona unui simplu constructor de ecrane izolate spre zona unui sistem care poate descrie un mic site sau o mică aplicație cu mai multe puncte de intrare.

#### 5.7.6 Preview-ul interactiv

Pe lângă editorul propriu-zis, aplicația include și o pagină de preview separată (‘CanvasPreview-Page’), folosită atât de autorul proiectului, cât și de vizitatorii care accesează un proiect public. În loc să redea direct structura internă a canvas-ului, această pagină reconstruiește reprezentarea intermediară pentru pagina selectată și cere backend-ului generarea HTML/CSS. Rezultatul este injectat într-un ‘iframe’ prin ‘srcdoc’, ceea ce permite observarea produsului într-un mediu mai apropiat de randarea finală decât suprafața de editare.

Preview-ul nu este doar o captură statică. El permite schimbarea paginii curente, selectarea unui cadru de vizualizare, redimensionarea viewport-ului și navigarea între pagini prin link-uri interne, interceptate prin ‘postMessage’ între ‘iframe’ și aplicație. În plus, pentru proiectele publice, aceeași pagină expune acțiuni de tip like, star și fork. Din acest motiv, preview-ul funcționează și ca punte între editor, conversie și partea socială a produsului.

#### 5.7.7 Persistare și integrare cu restul aplicației

Editorul nu este un spațiu închis. El comunică permanent cu serviciile de proiect, cu zona de preview și cu motorul de conversie. ‘CanvasPersistenceService’ este legătura principală dintre starea editorului și backend. ‘CanvasGenerationService’ este puntea spre export. ‘CanvasAi-ChatPersistenceService’ și istoricul extind editorul dincolo de o simplă editare de suprafață.

Din punct de vedere practic, această integrare a fost mai grea decât scrierea unui editor minimal. Nu era suficientă mutarea unui element pe ecran; era necesară și posibilitatea de salvare, validare, export și reîncărcare a rezultatului fără inconsistențe majore.

### 5.7.8 Persistența locală a conversației AI

O extensie utilă a editorului este păstrarea locală a conversației AI și a contextului asociat. ‘CanvasAiChatPersistenceService’ folosește ‘IndexedDB’ pentru a salva, per proiect, mesajele recente, promptul aflat în lucru, modelul selectat și tab-ul activ al interfeței. Persistarea este făcută cu debounce de 300 ms, iar la restaurare sunt acceptate doar intrările mai noi de 7 zile. În plus, serviciul limitează mesajele păstrate la 40 și normalizează lungimea textelor, pentru a evita acumularea necontrolată de date în browser.

Această decizie completează bine autosave-ul clasic al designului. Documentul proiectului rămâne în backend, pentru a putea fi sincronizat și exportat, în timp ce conversația AI și contextul ei imediat rămân locale, ceea ce reduce trafic inutil și permite reluarea rapidă a sesiunii după reîncărcarea editorului. Împreună cu restaurarea istoricului și cu flush-ul la ieșire din pagină, rezultă un comportament mult mai apropiat de un instrument de lucru continuu decât de un simplu demo de generare.

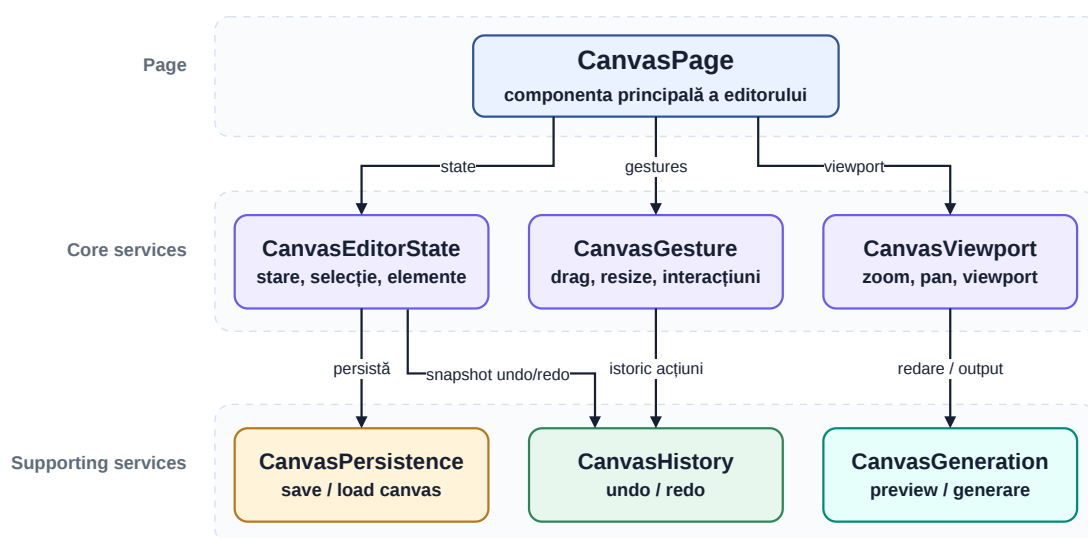


Figura 5.2: Servicii principale implicate în funcționarea editorului canvas

## 5.8 Decizii de implementare cu impact major

Privind la implementarea efectivă, există câteva decizii care au avut impact direct asupra stabilității și clarității codului.

Prima este faptul că logica specifică canvas-ului a fost păstrată în interiorul feature-ului ‘canvas’. În multe proiecte, utilitățile și serviciile migrează rapid spre zone generice, dar aici asta ar fi redus claritatea.

A doua este persistarea automată cu retry și flush controlat. Nu este o funcție spectaculoasă în demo, dar este foarte importantă în utilizare reală.

A treia este tratarea editorului ca pe o sumă de servicii coordonate, nu ca pe o singură componentă imensă care face totul. Deși ‘CanvasPage’ rămâne o componentă mare, mare parte din comportament este împinsă în servicii.

A patra este integrarea atentă dintre proiecte, thumbnail-uri și asset-uri. Fără această zonă, editorul ar fi avut o viață scurtă în afara unei sesiuni locale.

## 5.9 Concluzii intermediare

Implementarea platformei arată că Favigon este mai mult decât un proof of concept pentru generare AI. Autentificarea, proiectele, profilurile, funcțiile sociale și editorul canvas formează împreună baza produsului. În această bază, editorul este modulul cel mai complex, dar el nu ar avea aceeași valoare fără restul infrastructurii de produs.

Capitolul următor tratează componenta care diferențiază cel mai mult proiectul din punct de vedere tehnic: generarea asistată de AI și conversia interfețelor în cod.

## Capitolul 6

# Generarea asistată de AI și conversia interfețelor în cod

### 6.1 De ce nu este suficientă generarea directă

Una dintre ideile tentante atunci când lucrezi cu modele generative este să ceri direct rezultatul final: “generează-mi o pagină” sau chiar “generează-mi codul pentru această pagină”. Pentru demonstrații scurte, abordarea funcționează uneori surprinzător de bine. Totuși, pentru un produs software real, ea are câteva probleme serioase.

Prima problemă este controlul redus. Dacă modelul returnează direct cod, devine mai greu de separat unde s-a produs eroarea: în înțelegerea intenției, în structurarea layout-ului sau în alegerea stilurilor. A doua problemă este validarea. Codul poate arăta plauzibil fără să respecte constrângeri importante ale produsului. A treia problemă este reutilizarea. Dacă fiecare rezultat vine direct ca ieșire finală, nu există neapărat un nivel intern stabil pe care să îl poți edita și exporta în mai multe direcții.

Din aceste motive, în Favigon generarea asistată de AI este tratată ca un pipeline compus din etape distincte, nu ca pe un apel singular. În implementare, acest pipeline este orchestrat de ‘AiPipelineService’, iar integrarea cu modelul se face prin ‘OpenAiClient’.

### 6.2 Principiul pipeline-ului în trei faze

Soluția implementată în proiect împarte generarea în trei faze:

1. **intenție**: înțelegerea cererii și extragerea unei descrieri structurate a paginii;
2. **structură**: construirea arborelui intermediar de noduri, layout-uri și dimensiuni;
3. **stilizare**: aplicarea deciziilor vizuale pe structura deja generată.

Această împărțire pare simplă, dar în practică aduce mai multe avantaje. Fiecare etapă are un contract mai clar. Fiecare etapă poate fi testată sau inspectată separat. Fiecare etapă poate fi oprită anticipat dacă utilizatorul vrea doar o schiță. În plus, dacă rezultatul unei etape este invalid, eroarea este mai ușor de localizat. Fluxul general al acestei orchestrări este sintetizat în Figura 6.1.



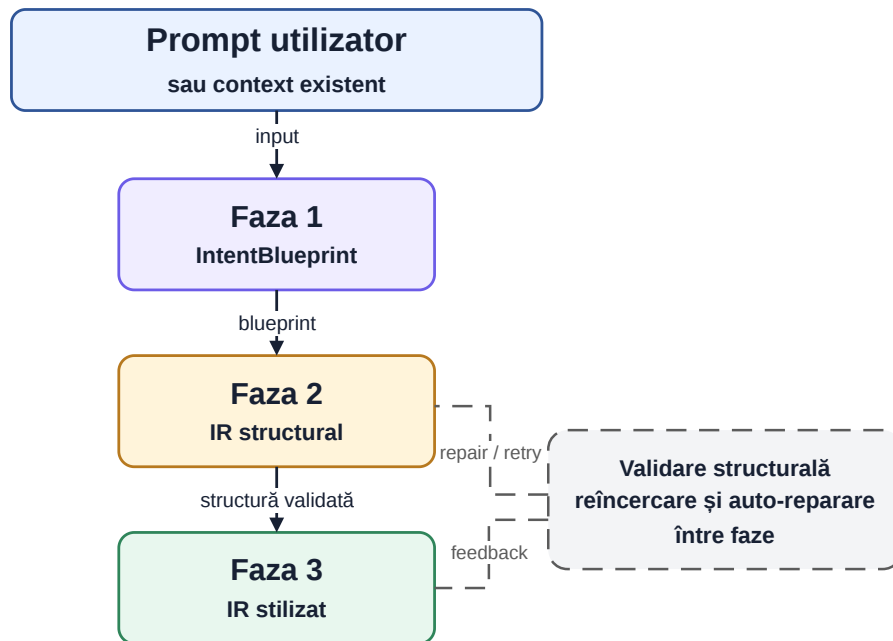


Figura 6.1: Pipeline-ul AI în trei faze folosit în Favigon

### 6.3 Faza 1: extragerea intenției

În prima etapă, sistemul nu încearcă să deseneze nimic și nu generează cod. Obiectivul este mai restrâns: să transforme promptul utilizatorului într-un *IntentBlueprint*. În implementare, acest blueprint conține informații precum tipul paginii, starea vizuală generală, personalitatea brandului, publicul țintă, CTA-ul principal și lista secțiunilor în ordinea lor.

Acest pas explicit este important din două motive. În primul rând, el forțează modelul să ia o decizie de structură înainte să intre în detalii estetice. În al doilea rând, oferă frontend-ului și backend-ului o reprezentare intermediară ușor de afișat, inspectat sau discutat.

În practică, această etapă seamănă mai mult cu o analiză de cerințe decât cu o generare vizuală. Promptul este reinterpretat într-un format intern. Dacă utilizatorul cere, de exemplu, o pagină de prezentare pentru un produs SaaS, rezultatul nu trebuie să fie imediat un hero cu gradient, ci o listă de secțiuni și intenții: navbar, hero, beneficii, social proof, CTA, footer și așa mai departe.

### 6.4 Faza 2: generarea structurii

A doua etapă este cea care construiește efectiv arborele intermediar de noduri și reprezintă etapa centrală a întregii abordări. În loc să fie cerute modelului HTML final sau componente de framework, sarcina transmisă este producerea unui arbore 'IRNode' cu layout, dimensiuni, padding, gap, poziționare și text.

Mai concret, structura generată include tipuri de noduri, copii, meta-informații, stiluri și reguli de layout. Promptul sistem pentru această fază este mult mai strict decât în prima etapă și include reguli explicite despre cum trebuie dimensionate secțiunile, cum trebuie folosite tipografiile, ce înseamnă 'fit-content', când se poate folosi '100'.

Această rigiditate este intenționată. Dacă în faza de intenție modelul are libertate mai mare, în faza structurală libertatea trebuie redusă. Altfel, ar exista prea multe variații greu de validat și greu de convertit.

## 6.5 Faza 3: stilizarea

A treia etapă pornește de la structura deja creată și aplică sistemul vizual: culori, gradient, umbre, raze, borduri, cursor și alte detalii de prezentare. În implementare, promptul pentru această fază conține o regulă foarte importantă: layout-ul și structura nu trebuie modificate. Altfel spus, stilizarea nu are voie să rescrie ceea ce a rezolvat deja etapa de structură.

Această separare între structură și stil este utilă atât pentru control, cât și pentru depanare. Dacă o pagină este logică, dar arată slab, știu că problema este în faza a treia. Dacă layout-ul este greșit, știu că problema este în faza a doua. În plus, această abordare seamănă mai bine cu modul în care lucrează și oamenii: întâi decid structura și ierarhia, apoi rafinează aspectul vizual.

## 6.6 Integrarea cu OpenAI API

În Favigon, comunicarea cu modelul este abstractizată prin 'OpenAIClient'. Acesta folosește endpoint-ul de *Chat Completions* și suportă atât răspuns obișnuit, cât și streaming [23]. Alegerea de a ascunde apelurile HTTP într-un adaptor de infrastructură păstrează serviciile de aplicație concentrate pe logică și face mai ușoară înlocuirea sau extinderea integrării în viitor.

Clientul construiește payload-ul cu mesaj de sistem, mesaj de utilizator, model configurabil și, unde este cazul, `response_format` bazat pe schemă JSON. În plus, pentru scenariile de streaming, clientul parcurge liniile SSE și extrage incremental conținutul transmis de model. În implementarea actuală, identificatorul modelului este citit din configurație, cu fallback la `gpt-5.4-mini`, iar cererile trimit atât `response_format`, cât și un prag explicit de `max_completion_tokens` pentru a limita comportamentul apelului la parametri observați și reproductibili.

Păstrarea identificatorului modelului în configurație, nu hardcodat în logică, este importantă deoarece partea experimentală a aplicației poate necesita ajustări de model în timp, iar aceste schimbări nu ar trebui să implice recompilarea serviciului de aplicație.

## 6.7 Structura cererilor AI și feedback incremental

Din punctul de vedere al API-ului, integrarea AI nu primește doar un simplu șir de text. Cererea de bază către endpoint-urile de design include promptul, un viewport țintă, modelul ales și, opțional, o structură IR existentă. Acest ultim câmp este important deoarece permite scenarii în care utilizatorul nu cere generarea unei pagini de la zero, ci rafinarea sau extinderea unei structuri deja existente. În cazul pipeline-ului pe trei faze, aceeași cerere mai include și parametrul `StopAfterPhase`, ceea ce face posibilă oprirea controlată după faza de intenție sau după faza structurală.

**Tabela 6.1:** Câmpuri relevante din cererile AI folosite de backend

| Câmp           | Semnificație                                  | Rol în procesare                                                           |
|----------------|-----------------------------------------------|----------------------------------------------------------------------------|
| Prompt         | descrierea textuală a interfeței dorite       | punctul de pornire pentru interpretarea intenției                          |
| ExistingIr     | structură intermediară existentă, opțională   | permite extindere, reparare sau rafinare în loc de generare complet nouă   |
| ViewportWidth  | lățimea de referință a paginii generate       | influențează presupunerile de layout și scară                              |
| Model          | identificatorul modelului folosit             | permite alegerea explicită între variantele configurate                    |
| StopAfterPhase | limită de oprire pentru pipeline-ul în 3 faze | utilă pentru schițe parțiale, depanare și observarea etapelor intermediare |

Suportul pentru streaming schimbă semnificativ și experiența utilizatorului. În loc să aștepte un singur răspuns final, frontend-ul poate primi evenimente incremental și poate afișa progresul procesării. În implementarea actuală, endpoint-urile de streaming trimit răspunsuri de tip `text/event-stream`, iar backend-ul emite linii SSE în care tipul evenimentului și conținutul sunt transmise separat. Pentru pipeline-ul în trei faze, evenimente precum `phase_start`, `phase_complete`, `result` sau `error` oferă o vizibilitate mult mai bună asupra procesului decât o simplă animație de încărcare.

**Listing 6.1:** Exemplu simplificat de evenimente SSE trimise în timpul pipeline-ului AI

```

event: phase_start
data: {"phase":1,"name":"intent"}

event: phase_complete
data: {"phase":1,"name":"intent"}

event: result
data: {"success":true,"stopAfterPhase":3}

```

Din punct de vedere operațional, fluxul complet dintre utilizator, frontend, controller, orchestration service și modelul AI este mai bine surprins printr-o diagramă de secvență. Figura 6.2 evidențiază atât schimburile principale de mesaje, cât și faptul că pipeline-ul nu este un singur apel, ci o succesiune controlată de faze cu feedback incremental.

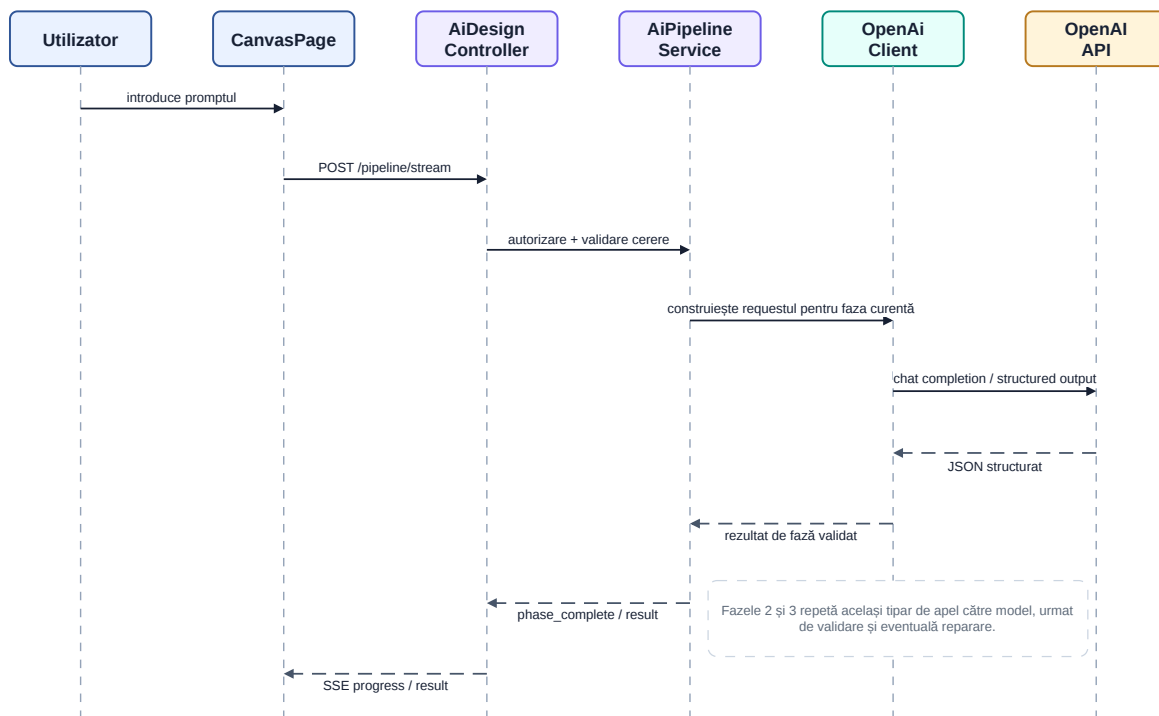


Figura 6.2: Diagrama de secvență simplificată pentru fluxul AI cu streaming

## 6.8 Rolul ieșirilor structurate

OpenAI documentează explicit posibilitatea obținerii unor ieșiri structurate, conforme cu un anumit format sau cu o anumită schemă [24]. În proiect, această capabilitate este foarte importantă. Dacă modelul ar întoarce doar text liber, parser-ul și validarea ar deveni mult mai fragile. În schimb, cu o schemă bine definită, aplicația poate detecta mai repede abaterile și poate decide dacă rezultatul este utilizabil sau nu.

Totuși, experiența din implementare arată că *structured output* nu este o soluție magică. El ajută la format, dar nu garantează automat calitatea semantică a rezultatului. De exemplu, un JSON perfect valid poate descrie o structură slabă sau o ierarhie de secțiuni nepotrivită. Din acest motiv, 'AiPipelineService' folosește atât deserializare și validare structurală, cât și reguli suplimentare și un pas de auto-reparare când rezultatul nu este acceptabil.

## 6.9 Mecanisme de validare și reparare

După fiecare etapă relevantă, rezultatul este analizat înainte să fie acceptat. Dacă în faza structurală arborele IR nu poate fi parsat sau nu respectă schema așteptată, serviciul încearcă un pas suplimentar de reparare. Acest pas trimite către model erorile de validare și o variantă trunchiată a rezultatului defect, cerând un JSON corectat.

Practic, modelul este tratat nu doar ca generator, ci și ca potențial asistent pentru repararea propriilor ieșiri. Această idee este utilă în contextul lucrării deoarece arată un mod mai realist de folosire a AI-ului: prima ieșire nu este presupusă perfectă, iar sistemul este proiectat explicit pentru a gestiona erori.

În plus, atât 'AiDesignService', cât și 'AiPipelineService' folosesc cache cu TTL de 10 minute pentru cereri identice și validează separat răspunsurile obținute. În varianta de pipeline există și posibilitatea de oprire după faza 1 sau 2, respectiv emiterea de evenimente SSE de tip

phase\_start și phase\_complete, ceea ce face procesul mai ușor de urmărit în interfață și în depanare.

## 6.10 Reprezentarea intermediară în detaliu

Structura 'IRNode' este elementul care leagă editorul, AI-ul și convertorul. Un nod include identificator, tip, proprietăți, layout, stil, poziționare, copii, variante responsive, metadate și, opțional, efecte de animație. Această structură este suficient de generală pentru a descrie atât containere, cât și text, imagini sau frame-uri de pagină.

În practică, modelul intermediar oferă trei beneficii majore.

Primul este **independența față de framework**. Un 'IRNode' nu este nici componentă Angular, nici JSX React, nici HTML final. De aceea poate fi transformat în toate aceste forme.

Al doilea este **validabilitatea**. Pentru că structura este explicită, poate fi verificată existența anumitor câmpuri, coerența layout-ului sau compatibilitatea dimensiunilor înainte de export.

Al treilea este **trasabilitatea**. Când ceva nu merge bine în export, structura intermediară poate fi inspectată și se poate vedea dacă problema vine din editor, din AI sau din convertor.

## 6.11 Maparea dintre canvas și IR

Pe partea de frontend, designul construit în editor nu este trimis direct către convertor în forma nativă a canvas-ului. În schimb, există mapper-e care transformă datele editorului în IR și invers. Acest detaliu este foarte important pentru arhitectura proiectului.

Dacă editorul ar fi legat direct de codul final, orice schimbare în convertor ar risca să rupă și logica din canvas. Prin introducerea mapper-elor, este creat un punct de decuplare. Editorul poate evolua într-o anumită direcție, iar convertorul în alta, atâta timp cât contractul IR rămâne coerent.

Maparea inversă, din IR în elemente de canvas, este la fel de utilă. Ea face posibilă integrarea rezultatelor generate de AI direct în editor, astfel încât utilizatorul să nu primească un export static, ci un design pe care îl poate modifica în continuare.

## 6.12 Exemplu concret de flux de la prompt la IR și export

Pentru a înțelege mai clar legătura dintre subsisteme, este util un exemplu simplificat. Presupunând un prompt de tipul „creează o pagină landing pentru o aplicație de task management, cu hero, beneficii și call-to-action”, prima fază nu produce HTML și nici noduri finale, ci o descriere structurată a intenției: tip de pagină, audiență, ton vizual și secțiuni în ordinea așteptată. A doua fază transformă această intenție într-un arbore IR cu frame principal, containere, text și butoane, iar a treia aplică stilurile. Abia după aceste etape structura ajunge la convertor.

În etapa de export, convertorul primește fie un singur IRNode, fie o listă de pagini. Pentru cazul simplu single-page, ConverterController apelează validarea și apoi Generate. Pentru cazul multi-page, aceeași rută de generare poate primi Pages, iar endpoint-ul dedicat pentru fișiere folosește GenerateMultiPage. Rezultatul nu este doar o bucată de cod brut, ci un set coerent de artefacte adaptate framework-ului cerut: HTML și CSS pentru varianta statică, respectiv fișiere de componentă, stil și structură pentru React sau Angular.

Acest exemplu arată de ce separarea pe etape este importantă. Dacă problema apare în secționarea paginii sau în ierarhia elementelor, ea poate fi localizată înainte de convertor. Dacă structura este corectă, dar codul rezultat nu respectă așteptările, zona de investigație se mută în motorul de export. În practică, această trasabilitate reduce mult costul depanării și face sistemul mai explicabil în comparație cu o abordare directă „prompt în, cod afară”.

## 6.13 Motorul de conversie

### 6.13.1 Separarea în proiect dedicat

Motorul de conversie este implementat în proiectul 'Favigon.Converter'. El este separat explicit de 'Favigon.Application' deoarece are propria logică de validare, transformare și emitere. Dacă ar fi rămas îngropat în același serviciu cu restul aplicației, ar fi fost mai greu de testat și mai greu de extins.

### 6.13.2 Validarea

Înainte de generare, 'ConverterService' cere motorului verificarea structurii IR. Dacă apar erori, ele sunt raportate și procesul se oprește. Această etapă este necesară pentru că ieșirea finală ar deveni altfel imprevizibilă. Este preferabil refuzul unui export decât generarea de cod inconsistent fără semnalarea problemei.

### 6.13.3 Generarea single-page și multi-page

'ConverterEngine' suportă atât generarea pentru o singură pagină, cât și generarea pentru mai multe pagini. Pentru multi-page, motorul produce seturi de fișiere și gestionează numele de pagini, slug-urile și structura finală. În cazul exportului Angular sau React, el generează și fișiere de proiect minimal, rute și fișiere CSS.

Această capacitate este importantă deoarece proiectul nu se reduce la o singură artboard. Odată ce editorul suportă mai multe pagini, exportul trebuie să poată păstra această structură.

### 6.13.4 Responsive output

O componentă mai tehnică și mai interesantă este exportul responsive. În loc să dublez pur și simplu tot arborele pentru fiecare breakpoint, motorul produce artefactele principale și apoi diferențe CSS pentru breakpoints suplimentare. În testele existente se vede că sunt tratate cazuri precum noduri prezente doar pe anumite breakpoint-uri, schimbări de ordine pentru copii flex și diferențe de pseudo-clase sau keyframes.

Aceasta este una dintre funcțiile care arată cel mai clar că proiectul nu se oprește la un export superficial. Există o preocupare reală pentru cum este construit rezultatul final și pentru cât de mult poate fi controlat. Fluxul complet canvas → IR → artefacte de export este rezumat în Figura 6.3.

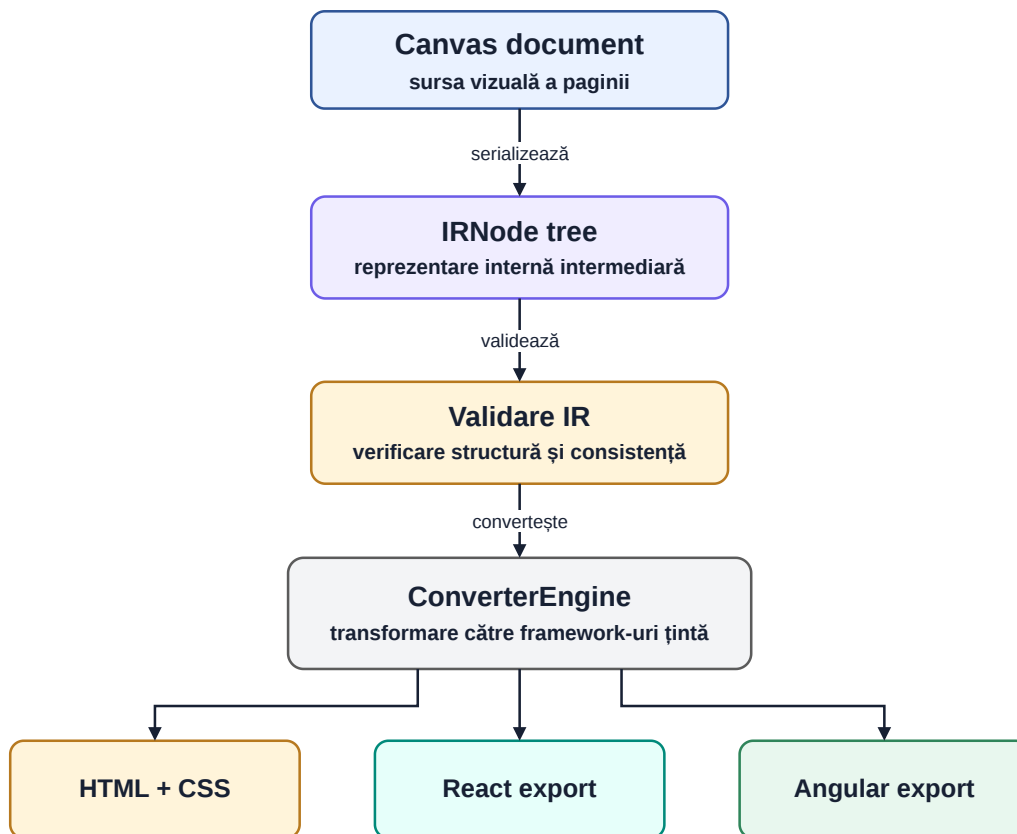


Figura 6.3: Fluxul design-to-code bazat pe reprezentarea intermediară

## 6.14 Legătura dintre AI și convertor

Un aspect foarte important este faptul că AI-ul și convertorul nu sunt două subsisteme complet separate. Ele comunică prin aceeași reprezentare intermediară. Fără această alegere, rezultatul generat de AI ar fi fost mult mai greu de transformat în cod într-un mod controlat.

Pe scurt, pipeline-ul AI generează un arbore care este suficient de bogat pentru a fi editat și suficient de formal pentru a fi convertit. În momentul în care această condiție este îndeplinită, editorul devine interfața de rafinare, iar convertorul devine ieșirea tehnică. Această relație dintre cele trei module — AI, editor și convertor — reprezintă contribuția tehnică principală a întregului proiect.

## 6.15 Limitări actuale

Deși soluția funcționează și este coerentă, există și limite.

În primul rând, calitatea rezultatului AI depinde încă mult de prompt și de regulile incluse în mesajele de sistem. Cu alte cuvinte, caracterul probabilistic al modelului nu este eliminat; el este doar controlat mai bine.

În al doilea rând, reprezentarea intermediară este suficient de bogată pentru scopurile actuale, dar nu acoperă toate situațiile pe care le-ar avea un constructor comercial matur. De exemplu, colaborarea în timp real, constrângerile mai avansate de componentizare sau integrarea automată cu design systems externe ar necesita extinderi suplimentare.

În al treilea rând, exportul generat este orientat spre claritate și portabilitate, nu neapărat spre cele mai sofisticate convenții ale fiecărui ecosistem. Pentru o platformă academică, acest compromis este acceptabil.

## **6.16 Concluzii intermediare**

Capitolul de față descrie partea cea mai originală din proiect. Generarea asistată de AI nu este tratată ca un artificiu separat, ci ca parte a unei arhitecturi care include reprezentare intermediară, validare, reparare și conversie în cod. Tocmai această integrare controlată face diferența între o demonstrație izolată și un produs software coerent.

În capitolul următor sunt discutate măsurile de securitate, testarea și modul în care a fost validat practic faptul că aceste module lucrează împreună într-o manieră suficient de stabilă.



## Capitolul 7

# Securitate, testare și validare

## 7.1 Rolul acestei etape în proiect

Într-un proiect de tip Favigon, securitatea și validarea nu pot fi tratate ca pași de final adăugați formal. Aplicația gestionează autentificare, sesiuni, proiecte private, upload de fișiere și integrare cu servicii externe costisitoare. În același timp, editorul și pipeline-ul AI au suficientă complexitate încât să necesite un efort real de verificare.

Din acest motiv, validarea a fost privită pe trei niveluri:

- **măsuri preventive în implementare**, care reduc suprafața de risc;
- **testare automată**, concentrată în special pe backend și convertor;
- **validare funcțională**, prin scenarii operaționale și fluxuri reale în interfață.

## 7.2 Măsuri de securitate implementate

### 7.2.1 Autentificare și sesiune

Securitatea începe în mod firesc din zona de autentificare. În Favigon, autentificarea se bazează pe JWT bearer validat în backend, dar transportat către client prin cookie HTTP only. Refresh token-ul este separat și limitat la un path specific. Această alegere reduce expunerea directă a token-urilor în codul client și simplifică rotația lor la refresh [7, 25].

Pentru scenariile de risc mai mare, proiectul include și autentificare în doi pași. Această măsură crește costul unui atac bazat exclusiv pe compromiterea parolei [26]. Chiar dacă implementarea actuală folosește cod prin email și nu un factor hardware, ea rămâne o îmbunătățire reală față de autentificarea cu un singur factor.

### 7.2.2 Limitarea expunerii API-ului

O altă măsură importantă este rate limiting-ul configurat diferențiat pe categorii de endpoint-uri. Zona de autentificare are praguri mai stricte, zona AI este limitată pentru a controla costul și abuzul, iar anumite endpoint-uri de utilizatori au propriile reguli. Documentația ASP.NET Core recomandă această abordare pentru protejarea aplicațiilor expuse public [18]. În configurația curentă, politicile observabile în 'Program.cs' sunt: 'auth' 10 cereri / minut, 'ai' 10 cereri / minut, 'converter' 30 cereri / minut cu coadă de 2 cereri și 'users' 60 cereri / minut.

În plus, backend-ul impune limite pentru dimensiunea cererilor. Cererile JSON globale au limită de 1 MB, iar endpoint-urile pentru upload sau flush folosesc praguri dedicate de 5 MB, 10 MB și 15 MB, în funcție de tipul operației. Este o măsură simplă, dar utilă împotriva folosirii necontrolate a resurselor.

### 7.2.3 Middleware și antete de securitate

În 'Program.cs', backend-ul include middleware personalizat pentru tratarea excepțiilor și middleware pentru antete de securitate. În mediul non-development sunt activate HSTS și redirecționarea HTTPS. CORS este configurat explicit pe baza unei liste de origini permise. Aceste decizii nu fac aplicația invulnerabilă, dar indică o preocupare practică pentru comportamentul în producție [20, 21].

## 7.2.4 Upload-uri și asset-uri

Proiectul permite încărcarea de imagini pentru profil și pentru proiecte. Din acest motiv, fișierele primite nu sunt tratate ca date “neutre”. Backend-ul validează tipul, dimensiunea și contextul în care upload-ul este permis. În plus, la actualizarea designului sunt identificate și șterse asset-urile orfane, tocmai pentru a evita acumularea de fișiere inutile.

**Tabela 7.1:** Măsurile de securitate implementate în Favigon

| Zonă              | Măsură implementată                              | Rol practic                                                     |
|-------------------|--------------------------------------------------|-----------------------------------------------------------------|
| Autentificare     | JWT în cookie HTTP only + re-fresh token separat | reduce expunerea token-urilor și permite reînnoirea sesiunii    |
| 2FA               | cod temporar prin email                          | ridică nivelul de protecție la login și operații sensibile      |
| API public        | rate limiting pe politici distincte              | reduce abuzul și protejează resursele costisitoare              |
| Transport         | HSTS și HTTPS în producție                       | reduce riscul de trafic neprotejat                              |
| Browser/API       | CORS configurat explicit                         | limitează originile care pot face cereri autentificate          |
| Upload-uri        | validare de dimensiune și tip                    | reduce riscul de folosire abuzivă a endpoint-urilor de fișiere  |
| Tratarea erorilor | middleware dedicat pentru excepții               | uniformizează răspunsurile și evită scurgeri inutile de detalii |

Tabelul 7.1 rezumă măsurile principale, însă valoarea lor practică apare tocmai din combinația lor: autentificare bazată pe cookie HTTP only, rotație separată prin refresh token, politici de limitare a cererilor și restricții explicite pentru upload-uri.

## 7.3 Strategia de testare

### 7.3.1 Testare automată backend

La data de 25 mai 2026, rularea locală a comenzii ‘dotnet test’ pentru proiectul ‘Backend/Favigon.Tests’ a raportat 108 teste trecute, 0 eșuate și o durată totală de aproximativ 5 secunde. Testele acoperă servicii, controllere, middleware și convertorul. Acest fapt este important pentru că zonele cele mai sensibile tehnic ale aplicației se află tocmai în backend și în motorul de conversie.

Mai precis, există teste pentru:

- autentificare: login, register, refresh, resetare parolă, 2FA;
- proiecte: controller și service;
- utilizatori: controller și service;
- convertor: controller și motorul propriu-zis;
- middleware: tratarea excepțiilor și antetele de securitate.

**Tabela 7.2:** Distribuția testelor automate backend

| <b>Fișier / zonă</b>                                                    | <b>Ce verifică în principal</b>                                    | <b>Nr. teste</b> |
|-------------------------------------------------------------------------|--------------------------------------------------------------------|------------------|
| AuthServiceLoginTests, Register, Refresh, PasswordManagement, TwoFactor | fluxurile esențiale de autentificare și recuperare cont            | 25               |
| ProjectServiceTests                                                     | gestionare proiecte, fork, bookmark, like, upload logic            | 20               |
| ProjectsControllerTests                                                 | comportamentul endpoint-urilor pentru proiecte                     | 11               |
| UsersControllerTests și UserServiceTests                                | profil, follow, search, stele și operații auxiliare pe utilizatori | 17               |
| ConverterControllerTests și ConverterEngineTests                        | validare IR, export, responsive, multi-page                        | 19               |
| ExceptionHandlerMiddlewareTests și SecurityHeadersMiddlewareTests       | tratarea erorilor și antetele HTTP                                 | 16               |
| <b>Total</b>                                                            | <b>teste backend automate trecute</b>                              | <b>108</b>       |

### 7.3.2 Testarea convertorului

Convertorul merită menționat separat deoarece este una dintre zonele cele mai sensibile la regresii. Testele existente nu se opresc la cazul simplu de export pentru un singur ecran. Ele includ și cazuri pentru:

- normalizarea rădăcinii provenite din canvas;
- generarea de clase CSS distincte pentru nume duplicate;
- omiterea unor stiluri implicite nerelevante;
- propagarea modului de overflow;
- noduri exclusive pentru anumite breakpoint-uri;
- reordonarea copiilor în layout-uri flex;
- diferențe de pseudo-clase și keyframes în export responsive.

Această suită de teste este importantă deoarece confirmă faptul că motorul nu este doar un generator static. El tratează situații reale care apar atunci când aceeași pagină trebuie exportată în mai multe variante sau cu structuri ușor diferite pe breakpoint-uri diferite.

### 7.3.3 Scenarii reprezentative ancorate în teste

Dincolo de numărul total al testelor, este utilă și observarea unor scenarii concrete care există deja în suită. Acestea arată mai clar ce comportamente sensibile sunt verificate efectiv, nu doar enumerate generic. Exemplele sintetice sunt rezumate în Tabelul 7.3.

**Tabela 7.3:** Scenarii reprezentative verificate prin teste automate

| Zonă                | Scenariu testat                             | Comportament verificat                                                                                              |
|---------------------|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Autentificare 2FA   | login cu 2FA activ                          | este emis un token intermediar, se trimite codul pe email și sesiunea finală nu este acordată înainte de verificare |
| Autentificare 2FA   | verificarea codului valid de login          | token-urile finale sunt emise, iar starea temporară de challenge este ștearsă                                       |
| Persistență proiect | înlocuirea unui asset imagine               | asset-ul vechi nefolosit este eliminat, evitând acumularea de fișiere orfane                                        |
| Export multi-page   | normalizarea rădăcinii provenite din canvas | coordonatele specifice editorului nu sunt propagate incorect în CSS-ul paginii exportate                            |
| Export responsive   | reordonarea copiilor flex pe breakpoint     | diferențele de ordine sunt emise în media queries, fără duplicarea inutilă a întregii structuri                     |

### 7.3.4 Indicatori cantitativi observați

Pe lângă distribuția testelor din Tabelul 7.2, validarea curentă oferă și câteva valori concrete, extrase direct din rulările și configurațiile existente ale proiectului.

**Tabela 7.4:** Indicatori cantitativi observați în validarea curentă

| Indicator                     | Valoare observată                                                                                | Sursă                                                         |
|-------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Execuția suitei backend       | 108 / 108 teste trecute, 0 eșuate, aproximativ 5 s                                               | rulare 'dotnet test' în 'Backend/Favigon.Tests' la 25.05.2026 |
| Suprafața API expusă          | 6 controllere și 62 endpoint-uri anotate HTTP                                                    | inventar din controllerele API                                |
| Politici de rate limiting     | 4 politici distincte ('auth', 'ai', 'converter', 'users')                                        | configurarea din 'Program.cs'                                 |
| Persistare automată în editor | debounce de 500 ms, flush maxim 4 s, 4 reîncercări cu bază 2 s                                   | 'CanvasPersistenceService'                                    |
| Orchestrare AI                | cache TTL 10 minute, 3 faze, suport pentru oprire după faza 1 sau 2 și evenimente SSE de progres | 'AiDesignService' și 'AiPipelineService'                      |
| Limite pentru cereri          | 1 MB global pentru JSON și 5 / 10 / 15 MB pe endpoint-uri sensibile                              | 'Program.cs' și attribute 'RequestSizeLimit'                  |

### 7.3.5 Testare frontend

Pe partea de frontend, testarea automată este în acest moment mult mai redusă. În repository există doar un test minimal la nivelul aplicației, nu o suită comparabilă cu cea din backend. Este importantă precizarea explicită a acestui fapt, pentru că altfel s-ar crea impresia falsă că proiectul este testat uniform în toate straturile.

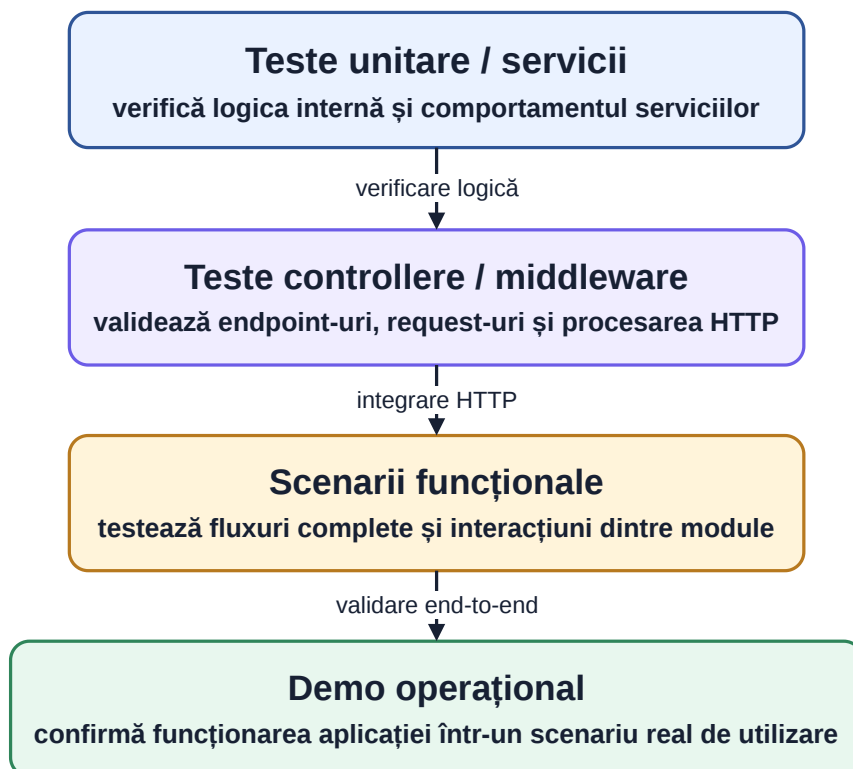
Motivul principal este complexitatea mare a editorului canvas și timpul limitat al proiectului. În locul unei suite largi, dar superficiale, efortul a fost direcționat spre stabilizarea logicii și spre validare funcțională directă. Nu este o soluție ideală pe termen lung, dar pentru stadiul actual al proiectului reprezintă un compromis explicit.

## 7.4 Validare funcțională

În completarea testelor automate, aplicația a fost verificată prin scenarii funcționale care urmăresc fluxurile principale ale produsului. Aceste scenarii sunt detaliate și în anexe, însă la nivel de sinteză ele acoperă:

1. creare cont, login și logout;
2. activare 2FA și autentificare cu pas suplimentar;
3. creare proiect nou și salvare automată;
4. editare în canvas cu mutare, redimensionare, selecție și text;
5. generare AI și integrarea rezultatului în editor;
6. export în HTML, React și Angular;
7. publicare, vizualizare publică, like, star și fork.

Această validare a fost utilă mai ales fiindcă a legat împreună subsisteme care, individual, puteau părea corecte. De exemplu, un export poate funcționa separat, dar dacă designul salvat nu este mapat corect în IR, fluxul cap-coadă devine fragil. La fel, 2FA poate funcționa la nivel de API, dar dacă interfața nu tratează bine starea intermediară, experiența utilizatorului se degradează rapid. Straturile complementare folosite pentru această verificare sunt reprezentate în Figura 7.1.



**Figura 7.1:** Straturi complementare de validare folosite în proiect

## 7.5 Limitări ale validării actuale

Deși proiectul are o bază bună de verificare, există și limite care trebuie menționate explicit.

Prima limită este testarea frontend-ului. Editorul canvas ar beneficia de teste automate mai bogate, în special pentru interacțiuni complexe și regresii vizuale.

A doua limită este partea de AI. Chiar dacă rezultatele pot fi validate structural, evaluarea calității estetice și a adecvării conținutului rămâne într-o anumită măsură dependentă de observația umană.

A treia limită este absența unor măsurători cantitative mai avansate pentru performanță sub sarcină. Proiectul are rate limiting și structură tehnică bună, dar nu a fost supus încă unei campanii complete de load testing.

## 7.6 Concluzii intermediare

Securitatea și validarea din Favigon nu rezolvă toate problemele posibile, dar oferă o bază credibilă pentru funcționarea produsului. Autentificarea, 2FA, limitarea cererilor, validarea upload-urilor și middleware-ul de protecție reduc riscuri concrete. În paralel, cele 108 teste backend și scenariile funcționale confirmă că aplicația nu este doar conceptual coerentă, ci și suficient de stabilă pentru demonstrare și utilizare controlată.

Pe baza acestei evaluări, capitolul final sintetizează rezultatele proiectului și direcțiile de dezvoltare viitoare.

## Capitolul 8

# Concluzii și perspective de dezvoltare

### 8.1 Gradul de îndeplinire a obiectivelor

Analiza din capitolele anterioare arată că obiectivul principal al lucrării a fost atins. A fost proiectată și implementată o platformă web care permite crearea și editarea vizuală a interfețelor, generarea asistată de AI, conversia în cod și partajarea proiectelor între utilizatori. În plus, aceste funcții nu există separat, ci sunt integrate într-un produs unitar.

Din obiectivele specifice formulate la început, cele mai importante au fost îndeplinite astfel:

- arhitectura full-stack este separată clar pe frontend, API, aplicație, infrastructură și convertor;
- editorul canvas suportă stări, gesturi, istoric, clipboard, persistare și lucru multi-page;
- reprezentarea intermediară permite decuplarea dintre editor, AI și export;
- pipeline-ul AI în trei faze funcționează și poate fi urmărit în mod controlat;
- exportul în HTML, React și Angular este operațional, inclusiv pentru scenarii multi-page și responsive;
- produsul include funcții sociale reale, nu doar componente tehnice izolate.

Din perspectiva unei lucrări de licență, este important și faptul că proiectul nu s-a blocat la nivel de demo local. Existența autentificării, a profilurilor, a proiectelor publice și a testării automate pe backend arată că soluția a fost tratată ca aplicație completă, nu doar ca experiment punctual. Trasabilitatea dintre obiective, module și validare este sintetizată deja în Tabelul 4.1, iar indicatorii cantitativi principali apar în Tabelul 7.4.

### 8.2 Puncte forte ale proiectului

Principalul punct forte al proiectului este coerența dintre subsisteme. Editorul, AI-ul, exportul și zona socială nu sunt doar funcții alăturate, ci lucrează pe același model de proiect și pe aceeași infrastructură de produs. Tocmai această integrare este contribuția cea mai valoroasă.

Un al doilea punct forte este reprezentarea intermediară. Fără ea, proiectul ar fi avut fie un editor izolat, fie un generator AI greu de controlat, fie un convertor legat rigid de o singură tehnologie. IR-ul a permis toate aceste funcții simultan.

Un al treilea punct forte este modul în care este introdus AI-ul. În loc ca modelul să fie tratat ca o sursă imprevizibilă de cod final, sunt introduse etape, constrângeri și validări. Rezultatul nu este perfect, dar este mult mai potrivit pentru o aplicație software decât un simplu apel direct din prompt în cod.

În fine, un punct forte este și faptul că backend-ul a fost testat consistent. Cele 108 teste automate nu acoperă întreaga aplicație, dar oferă încredere în zonele cele mai sensibile ale logicii de business și ale convertorului.

### 8.3 Limitări

Ca orice proiect de licență, Favigon are și limitări clare.

Prima limită este testarea frontend-ului, în special a editorului canvas. Deși acesta funcționează și a fost verificat funcțional, nu are încă o acoperire automată comparabilă cu backend-ul.

A doua limită este dependența de calitatea ieșirilor AI. Chiar dacă pipeline-ul și validarea reduc variabilitatea, rezultatul final depinde totuși de model și de prompt.

A treia limită este lipsa colaborării în timp real între utilizatori în interiorul aceluiași proiect. Pentru o platformă orientată și spre social și reutilizare, aceasta ar fi o direcție logică de extindere.

A patra limită este legată de rafinarea exportului. Deși motorul produce rezultate utile și coerente, proiectele generate ar putea fi îmbunătățite în viitor prin integrare mai profundă cu convențiile fiecărui framework sau cu design systems preexistente.

## **8.4 Perspective de dezvoltare**

Pornind de la aceste limite, văd mai multe direcții naturale de continuare.

### **8.4.1 Colaborare și versionare**

O primă direcție ar fi introducerea colaborării în timp real și a unei forme mai bogate de versionare. În prezent, proiectele pot fi persistate și fork-uite, dar nu există încă editare simultană sau comentarii direct în canvas. Acest pas ar apropia platforma de instrumentele moderne folosite în echipe.

### **8.4.2 Îmbunătățirea pipeline-ului AI**

O a doua direcție este rafinarea pipeline-ului AI. De exemplu, pot fi introduse exemple contextuale pe tipuri de pagini, ranking pentru rezultate alternative, evaluare automată a structurii sau suport pentru transformări iterative mai precise pornind de la un design existent.

### **8.4.3 Componentizare și design systems**

O a treia direcție este introducerea componentelor reutilizabile la nivel de editor și de export. În acest moment, structura este suficient de bogată pentru pagini și secțiuni, dar un nivel suplimentar de componentizare ar face proiectul mai potrivit pentru aplicații mai mari și pentru integrare cu biblioteci UI existente.

### **8.4.4 Testare frontend și observabilitate**

O a patra direcție, foarte importantă, este extinderea testării frontend și introducerea unei observabilități mai bune. Pentru editor ar fi utile teste end-to-end și teste vizuale automate. Pentru backend și pentru AI ar fi utilă și o zonă mai clară de metrics și logging operațional.

## **8.5 Încheiere**

Dincolo de rezultatul tehnic, proiectul a fost și un exercițiu de integrare. Au trebuit combinate decizii de arhitectură, experiență de utilizator, persistare, securitate, AI și export într-o singură aplicație. Nu toate zonele sunt la nivelul unui produs comercial matur, dar platforma demonstrează clar fezabilitatea unui astfel de flux integrat.

Lucrarea își atinge scopul tocmai prin faptul că nu prezintă doar o idee, ci un produs funcțional în care ideea poate fi observată concret. Favigon arată că un editor vizual, o reprezentare intermediară, un pipeline AI și un motor de conversie pot fi reunite într-o arhitectură coerentă și extensibilă. Din acest motiv, proiectul poate reprezenta atât un rezultat final valid pentru lucrarea de licență, cât și o bază reală pentru dezvoltări viitoare.



## Anexa A

# Scenarii de demo și validare funcțională

### A.1 Scopul anexei

Această anexă sintetizează scenariile considerate cele mai relevante pentru demonstrarea aplicației în fața comisiei și pentru validarea funcțională a fluxurilor principale. Ele sunt prezentate separat deoarece nu reprezintă doar o listă de pași de test, ci și un mod practic de a demonstra coerența produsului.

În toate scenariile urmăresc aceeași idee: nu este suficient ca un ecran să arate bine sau ca un endpoint să răspundă corect. Important este ca fluxul cap-coadă să fie stabil și ușor de explicat.

### A.2 Scenariul 1: creare cont și autentificare

**Tabela A.1:** Scenariul de bază pentru cont și autentificare

| Element     | Descriere                                                                 | Rezultat așteptat                                       |
|-------------|---------------------------------------------------------------------------|---------------------------------------------------------|
| Precondiții | utilizatorul nu are cont sau nu este autentificat                         | pagina de login este accesibilă                         |
| Pași        | creare cont, autentificare, logout, autentificare din nou                 | fluxul este complet și fără pierdere de sesiune         |
| Verificări  | cookie de autentificare, redirecționare, încărcarea utilizatorului curent | utilizatorul este trimis corect către profil / proiecte |

Acest scenariu este util în demo deoarece confirmă funcționarea infrastructurii de bază: API, cookies, load current user, redirect din rută și sincronizarea dintre frontend și backend. Chiar dacă pare un scenariu simplu, el validează o parte importantă din fundația produsului.

### A.3 Scenariul 2: activare și folosire 2FA

**Tabela A.2:** Scenariul pentru autentificare în doi pași

| Element     | Descriere                                                              | Rezultat așteptat                                         |
|-------------|------------------------------------------------------------------------|-----------------------------------------------------------|
| Precondiții | utilizator autentificat, adresă de email funcțională                   | poate solicita activarea 2FA                              |
| Pași        | cerere cod, confirmare cod, logout, login nou, verificare cod la login | autentificarea se finalizează doar după validarea codului |
| Verificări  | afișarea stării intermediare și emiterea cookie-urilor finale          | sesiunea finală apare doar după a doua etapă              |

În prezentarea live, acest scenariu este bun deoarece arată clar că proiectul include o măsură de securitate reală, nu doar autentificare simplă. În plus, evidențiază și un mic flux de stare intermediară în interfață, ceea ce este util de explicat în fața comisiei.

## A.4 Scenariul 3: creare proiect și editare în canvas

**Tabela A.3:** Scenariul principal de lucru în editor

| Element     | Descriere                                                                        | Rezultat așteptat                                      |
|-------------|----------------------------------------------------------------------------------|--------------------------------------------------------|
| Precondiții | utilizator autentificat                                                          | poate crea un proiect nou                              |
| Pași        | deschidere proiect, adăugare elemente, mutare, resize, editare text, pagină nouă | editorul rămâne fluid și starea se actualizează corect |
| Verificări  | selecție, zoom, istoric, persistare automată, thumbnail                          | la reîncărcare proiectul este recuperat corect         |

Acesta este scenariul central al demo-ului. Într-o prezentare foarte scurtă, el merită păstrat în mod obligatoriu. Demonstrează simultan editorul, persistarea, proiectele și calitatea interacțiunilor de bază.

## A.5 Scenariul 4: generare AI

**Tabela A.4:** Scenariul de generare asistată de AI

| Element     | Descriere                                                     | Rezultat așteptat                                  |
|-------------|---------------------------------------------------------------|----------------------------------------------------|
| Precondiții | proiect deschis în editor                                     | utilizatorul poate introduce un prompt             |
| Pași        | lansare generare, urmărire progres, import rezultat în canvas | sistemul produce o structură coerentă și editabilă |
| Verificări  | răspuns normal sau streaming, mapare în elemente de canvas    | rezultatul poate fi modificat manual ulterior      |

La demo, nu este obligatoriu să arăt toate fazele interne ale pipeline-ului, însă este foarte util să pot explica faptul că sistemul nu generează direct cod final, ci trece prin etape intermediare. Acest lucru diferențiază produsul de o simplă integrare superficială cu un model AI.

## A.6 Scenariul 5: export în cod

În prezentare, acest scenariu ar trebui arătat imediat după o editare sau după o generare AI, pentru a sublinia că platforma nu se oprește în editor. O secvență bună de demo este: modific proiectul, salvez, export și deschid rapid unul dintre fișierele generate.

## A.7 Scenariul 6: explorare, social și fork

Acest scenariu este valoros deoarece arată că proiectul nu este doar o unealtă personală. El oferă și o logică de circulație a conținutului între utilizatori, ceea ce îl apropie mai mult de o platformă.

**Tabela A.5:** Scenariul de export al proiectului

| Element     | Descriere                                                    | Rezultat așteptat                                       |
|-------------|--------------------------------------------------------------|---------------------------------------------------------|
| Precondiții | proiect cu structură validă                                  | utilizatorul alege framework-ul țintă                   |
| Pași        | validare, generare fișiere, inspec-tare rezultat             | sunt produse fișierele specifice HTML / React / Angular |
| Verificări  | coerența codului generat, exis-tența CSS-ului și a paginilor | exportul este reutilizabil ca punct de pornire          |

**Tabela A.6:** Scenariul de reutilizare socială

| Element     | Descriere                                               | Rezultat așteptat                                       |
|-------------|---------------------------------------------------------|---------------------------------------------------------|
| Precondiții | există proiecte publice în sistem                       | explore și profilurile publice sunt accesibile          |
| Pași        | vizualizare proiect public, like, star, fork            | proiectul este copiat în spațiul utilizatorului curent  |
| Verificări  | relația de fork și indicatorii sociali sunt actualizați | utilizatorul poate continua edita-rea fork-ului propriu |

**Recomandare pentru ordinea demo-ului.**

Într-o prezentare scurtă, o ordine eficientă este:

1. login rapid;
2. deschidere proiect existent;
3. editare scurtă în canvas;
4. generare AI pe o pagină sau secțiune;
5. export în cod;
6. prezentare scurtă a zonei explore și a unui fork.

Această succesiune este eficientă deoarece pornește de la ceva vizibil și interactiv, arată funcția diferențiată de AI, apoi închide bucla prin export și reutilizare.

# Anexa B

## Exemple tehnice și note de arhitectură

### B.1 Maparea dintre modulele proiectului

Tabela B.1: Maparea dintre proiectele din soluție și responsabilitățile lor

| Proiect / zonă         | Responsabilitate principală                                   |
|------------------------|---------------------------------------------------------------|
| Frontend               | interfață, rutare, editor canvas, preview, apeluri API        |
| Favigon.API            | endpoint-uri HTTP, middleware, configurare infrastructură web |
| Favigon.Application    | servicii de business, DTO-uri, validări și orchestrare logică |
| Favigon.Infrastructure | repository-uri, EF Core, integrare OpenAI, email, storage     |
| Favigon.Domain         | entități persistente și relațiile lor                         |
| Favigon.Converter      | reprezentare intermediară, validare și export în cod          |
| Favigon.Tests          | teste automate pentru backend și convertor                    |

### B.2 Exemplu simplificat de reprezentare intermediară

În listarea următoare prezint un exemplu simplificat de arbore IR folosit pentru descrierea unei secțiuni de tip hero. În implementarea reală, structura poate include mai multe câmpuri și mai multe variante responsive, însă ideea de bază rămâne aceeași: fiecare nod descrie tipul, dimensiunea, layout-ul și conținutul relevant.

Listing B.1: Exemplu simplificat de IR pentru o secțiune hero

```
{
  "id": "frame-home",
  "type": "Frame",
  "meta": { "name": "Home Page" },
  "style": {
    "width": { "value": 1280, "unit": "px" },
    "height": { "value": 0, "unit": "fit-content" }
  },
  "children": [
    {
      "id": "hero-section",
      "type": "Container",
      "meta": { "name": "Hero Section" },
      "layout": {
        "mode": "Flex",
        "direction": "Row",
        "gap": { "value": 48, "unit": "px" }
      },
      "style": {
        "padding": {
          "top": { "value": 96, "unit": "px" },
          "right": { "value": 80, "unit": "px" },
          "bottom": { "value": 96, "unit": "px" },

```

```

    "left": { "value": 80, "unit": "px" }
  },
  "children": [
    {
      "id": "hero-title",
      "type": "Text",
      "props": { "text": "șConstruiete ținterfee și ăexport-le în cod" }
    }
  ]
}
]
}

```

Acest exemplu arată de ce IR-ul este util. El nu conține numai text sau numai stil, ci și relațiile de ierarhie și layout. Astfel, același document poate fi randat într-un canvas, validat și apoi convertit într-o implementare concretă.

### B.3 Exemple simplificate de cereri către API

În această anexă este utilă și o vedere foarte concretă asupra formei cererilor transmise către backend pentru funcțiile cele mai relevante tehnic. Exemplele de mai jos nu reproduc toate câmpurile posibile din implementarea reală, dar păstrează structura esențială a payload-urilor.

**Listing B.2:** Exemplu simplificat de request pentru pipeline-ul AI

```

{
  "prompt": "Landing page pentru o aplicatie SaaS de task management",
  "viewportWidth": 1280,
  "model": "gpt-5.4-mini",
  "stopAfterPhase": 3,
  "existingIr": null
}

```

**Listing B.3:** Exemplu simplificat de request pentru generarea de cod

```

{
  "framework": "html",
  "ir": {
    "id": "frame-home",
    "type": "Frame",
    "children": []
  }
}

```

În practică, aceste două exemple ilustrează două tipuri foarte diferite de cereri. Prima cere procesare probabilistică, etapizată și validată semantic. A doua cere transformare deterministă dintr-o structură deja formalizată. Tocmai această separare este una dintre ideile arhitecturale importante ale proiectului.

### B.4 Exemplu simplificat de cod generat

**Listing B.4:** Exemplu scurt de HTML generat de convertor

```

<section class="hero-section">

```

```

<div class="hero-copy">
  <h1 class="hero-title">§Construiete țiinterfee și ăexport-le în cod</h1>
  <p class="hero-body">§Pornete de la un prompt, ărafineaz în editor și ăexport
    rezultatul.</p>
</div>
</section>

```

**Listing B.5:** Exemplu scurt de CSS generat de convertor

```

.hero-section {
  display: flex;
  flex-direction: row;
  gap: 48px;
  padding: 96px 80px;
}

.hero-title {
  font-size: 64px;
  font-weight: 800;
}

```

## B.5 Note practice despre organizarea editorului

În implementare, componenta 'CanvasPage' acționează ca punct de orchestrare. Totuși, comportamentul editorului este împărțit între servicii focalizate. Această decizie poate părea mai complicată la prima vedere, dar a adus trei avantaje clare:

1. comportamentele precum zoom, gesturi sau istoric au putut fi dezvoltate independent;
2. persistarea și exportul au rămas conectate la starea editorului fără să încarce excesiv componenta;
3. logica a devenit mai ușor de urmărit atunci când apăreau bug-uri pe zone specifice.

Pentru proiecte viitoare, această organizare face mai ușoară introducerea unor funcții precum comentarii pe canvas, componente reutilizabile, colaborare în timp real sau reguli de aliniere mai avansate.

## B.6 Exemplu simplificat de evenimente de streaming

În scenariile de generare incrementală, frontend-ul nu primește doar un răspuns final, ci o secvență de evenimente SSE. Varianta simplificată de mai jos redă forma protocolului folosit pentru notificarea progresului și pentru transmiterea rezultatului final.

**Listing B.6:** Exemplu simplificat de flux SSE pentru pipeline-ul AI

```

event: phase_start
data: {"phase":1,"name":"intent"}

event: phase_complete
data: {"phase":1,"name":"intent"}

event: phase_start
data: {"phase":2,"name":"structure"}

event: result

```

```
data: {"success":true,"stopAfterPhase":3}
```

Din punct de vedere al interfeței, acest format face posibilă afișarea unui progres real, nu doar a unei stări generice de așteptare. Din punct de vedere al depanării, el ajută la observarea etapei în care apare o eventuală eroare sau oprire prematură.

### **Compilarea documentației.**

Lucrarea este gândită să poată fi compilată local prin scriptul PowerShell din ‘docs/licenta’. Pașii executați sunt:

1. prima trecere ‘xelatex’, pentru fișiere auxiliare și structură;
2. ‘biber’, pentru procesarea bibliografiei;
3. încă două treceri ‘xelatex’, pentru rezolvarea referințelor, figurilor și citărilor.

Scriptul folosește această succesiune explicită deoarece în mediul local ‘latexmk’ nu poate fi folosit direct fără Perl. Din acest motiv, varianta manuală controlată este mai robustă pentru repository-ul actual.

## Anexa C

# Inventar de endpoint-uri API

### C.1 Observații generale

Această anexă sintetizează endpoint-urile principale ale aplicației. Nu urmăresc să refac documentația Swagger în întregime, ci să ofer o imagine clară asupra suprafeței API-ului și a modului în care aceasta este împărțită pe domenii funcționale.

### C.2 Endpoint-uri pentru cont și autentificare

**Tabela C.1:** Endpoint-uri principale din zona de autentificare

| Metodă | Rută                                      | Rol                                       |
|--------|-------------------------------------------|-------------------------------------------|
| POST   | ‘/api/account/register’                   | creare cont nou                           |
| POST   | ‘/api/account/login’                      | autentificare clasică                     |
| POST   | ‘/api/account/oauth2/github’              | autentificare prin GitHub                 |
| POST   | ‘/api/account/oauth2/google’              | autentificare prin Google                 |
| POST   | ‘/api/account/two-factor/verify-login’    | finalizarea login-ului cu 2FA             |
| POST   | ‘/api/account/oauth2/github/link’         | legare cont GitHub la utilizatorul curent |
| POST   | ‘/api/account/oauth2/google/link’         | legare cont Google la utilizatorul curent |
| POST   | ‘/api/account/forgot-password’            | inițiere resetare parolă                  |
| POST   | ‘/api/account/reset-password’             | setarea noii parole prin token            |
| POST   | ‘/api/account/set-password’               | adăugarea unei parole pentru cont OAuth   |
| POST   | ‘/api/account/change-password’            | schimbarea parolei                        |
| POST   | ‘/api/account/two-factor/enable/request’  | trimiterea codului pentru activare 2FA    |
| POST   | ‘/api/account/two-factor/enable/confirm’  | confirmarea activării 2FA                 |
| POST   | ‘/api/account/two-factor/disable/request’ | trimiterea codului pentru dezactivare 2FA |
| POST   | ‘/api/account/two-factor/disable/confirm’ | confirmarea dezactivării 2FA              |
| POST   | ‘/api/account/refresh’                    | reînnoirea token-ului de acces            |
| POST   | ‘/api/account/logout’                     | încheierea sesiunii                       |

### C.3 Endpoint-uri pentru proiecte



**Tabela C.2:** Endpoint-uri principale pentru proiecte

| Metodă | Rută                             | Rol                                     |
|--------|----------------------------------|-----------------------------------------|
| GET    | ‘/api/projects’                  | lista proiectelor utilizatorului curent |
| GET    | ‘/api/projects/id’               | proiect după identificator              |
| GET    | ‘/api/projects/by-slug/slug’     | proiect după slug                       |
| POST   | ‘/api/projects’                  | creare proiect                          |
| PUT    | ‘/api/projects/id’               | actualizare metadate proiect            |
| DELETE | ‘/api/projects/id’               | ștergere proiect                        |
| POST   | ‘/api/projects/id/assets/images’ | upload imagine pentru canvas            |
| GET    | ‘/api/projects/id/design’        | încărcare design salvat                 |
| PUT    | ‘/api/projects/id/design’        | salvare design                          |
| POST   | ‘/api/projects/id/flush’         | salvare combinată design + thumb-nail   |
| PUT    | ‘/api/projects/id/thumbnail’     | actualizare thumbnail                   |
| GET    | ‘/api/projects/user/userId’      | proiecte publice ale unui utilizator    |
| POST   | ‘/api/projects/id/star’          | bookmark / star                         |
| DELETE | ‘/api/projects/id/star’          | eliminare bookmark                      |
| POST   | ‘/api/projects/id/view’          | înregistrare vizualizare                |
| POST   | ‘/api/projects/id/like’          | apreciere proiect                       |
| DELETE | ‘/api/projects/id/like’          | eliminare apreciere                     |
| POST   | ‘/api/projects/id/fork’          | creare fork după proiect public         |

## C.4 Endpoint-uri pentru utilizatori și profiluri

**Tabela C.3:** Endpoint-uri principale pentru utilizatori

| Metodă | Rută                                     | Rol                                       |
|--------|------------------------------------------|-------------------------------------------|
| GET    | ‘/api/users/me’                          | profilul utilizatorului curent            |
| PUT    | ‘/api/users/me’                          | actualizare profil propriu                |
| POST   | ‘/api/users/me/profile-image’            | upload imagine de profil                  |
| DELETE | ‘/api/users/me’                          | ștergere cont propriu                     |
| DELETE | ‘/api/users/me/linked-accounts/provider’ | deconectare provider OAuth                |
| GET    | ‘/api/users/search?q=...’                | căutare de utilizatori                    |
| GET    | ‘/api/users/username’                    | profil public după username               |
| POST   | ‘/api/users/username/follow’             | urmărire utilizator                       |
| DELETE | ‘/api/users/username/follow’             | oprire urmărire                           |
| GET    | ‘/api/users/username/followers’          | lista urmăritorilor                       |
| GET    | ‘/api/users/username/following’          | lista utilizatorilor urmăriți             |
| GET    | ‘/api/users/me/stars’                    | proiectele salvate de utilizatorul curent |

## C.5 Endpoint-uri pentru explore și recomandări

**Tabela C.4:** Endpoint-uri din zona explore

| Metodă | Rută                       | Rol                                             |
|--------|----------------------------|-------------------------------------------------|
| GET    | ‘/api/explore/trending’    | proiecte populare                               |
| GET    | ‘/api/explore/recommended’ | recomandări de proiecte, eventual personalizate |
| GET    | ‘/api/explore/people’      | sugestii de utilizatori                         |

## C.6 Endpoint-uri pentru AI și conversie

**Tabela C.5:** Endpoint-uri pentru AI și converter

| Metodă | Rută                             | Rol                                       |
|--------|----------------------------------|-------------------------------------------|
| POST   | ‘/api/ai/design’                 | generare design într-o singură cerere     |
| POST   | ‘/api/ai/design/stream’          | generare design în regim SSE              |
| POST   | ‘/api/ai/design/pipeline’        | rulare pipeline AI în trei faze           |
| POST   | ‘/api/ai/design/pipeline/stream’ | rulare pipeline AI cu streaming           |
| POST   | ‘/api/converter/generate’        | export pentru o pagină sau set responsive |
| POST   | ‘/api/converter/validate’        | validare structură IR                     |
| POST   | ‘/api/converter/generate-files’  | generare fișiere pentru export multi-page |

### Observație finală.

Din inventarul de mai sus se vede că API-ul nu este limitat la o singură zonă funcțională. El trebuie să deservească atât fluxuri de produs clasice, cât și funcționalități tehnice speciale, cum sunt streaming-ul AI și exportul multi-framework. Tocmai această diversitate justifică structura pe straturi din backend și separarea convertorului în proiect propriu.

# Bibliografie

- [1] Angular Team, *Angular Routing*, 2026, URL: <https://angular.dev/guide/routing> (accesat în 23.5.2026).
- [2] Angular Team, *provideRouter*, 2026, URL: <https://angular.dev/api/router/provideRouter> (accesat în 23.5.2026).
- [3] Angular Team, *Signals — Overview*, 2026, URL: <https://angular.dev/guide/signals> (accesat în 23.5.2026).
- [4] Angular Team, *Standalone Components*, 2026, URL: <https://angular.dev/reference/migrations/standalone> (accesat în 23.5.2026).
- [5] Len Bass, Paul Clements și Rick Kazman, *Software Architecture in Practice*, a 4-a ed., Addison-Wesley, 2021.
- [6] Tony Beltramelli, „pix2code: Generating Code from a Graphical User Interface Screenshot”, în *arXiv preprint arXiv:1705.07962* (2017), URL: <https://arxiv.org/abs/1705.07962> (accesat în 23.5.2026).
- [7] Damien Bowden, *Configure JWT bearer authentication in ASP.NET Core*, 2025, URL: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/configure-jwt-bearer-authentication?view=aspnetcore-9.0> (accesat în 23.5.2026).
- [8] Builder.io, *Builder Visual Development Platform*, 2026, URL: <https://www.builder.io/visual-development-platform> (accesat în 23.5.2026).
- [9] Builder.io, *Code, create, iterate faster with AI*, 2026, URL: <https://www.builder.io/ai> (accesat în 23.5.2026).
- [10] Ravi Chugh et al., „Programmatic and Direct Manipulation, Together at Last”, în *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ACM, 2016, pp. 341–354, DOI: [10.1145/2908080.2908103](https://doi.org/10.1145/2908080.2908103), URL: <https://doi.org/10.1145/2908080.2908103> (accesat în 25.5.2026).
- [11] Cloudflare, *Cloudflare Tunnel*, 2026, URL: <https://developers.cloudflare.com/tunnel/> (accesat în 23.5.2026).
- [12] Docker, *Compose file reference*, 2026, URL: <https://docs.docker.com/compose/compose-file/> (accesat în 23.5.2026).
- [13] Docker, *Docker Compose*, 2026, URL: <https://docs.docker.com/compose/> (accesat în 23.5.2026).
- [14] Roy Thomas Fielding, „Architectural Styles and the Design of Network-based Software Architectures”, Teză de doct., University of California, Irvine, 2000, URL: [https://ics.uci.edu/~fielding/pubs/dissertation/fielding\\_dissertation.pdf](https://ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf) (accesat în 23.5.2026).
- [15] Figma, *Dev Mode: Design-to-Development*, 2026, URL: <https://www.figma.com/dev-mode/> (accesat în 23.5.2026).
- [16] Framer, *Wireframer*, 2026, URL: <https://www.framer.com/wireframer/> (accesat în 23.5.2026).
- [17] Ben Hutton et al., *JSON Schema Validation: A Vocabulary for Structural Validation of JSON*, 2022, URL: <https://json-schema.org/draft/2020-12/json-schema-validation> (accesat în 23.5.2026).
- [18] Arvin Kahbazi, Maarten Balliauw și Rick Anderson, *Rate limiting middleware in ASP.NET Core*, 2025, URL: <https://learn.microsoft.com/en-us/aspnet/core/performance/rate-limit?view=aspnetcore-10.0> (accesat în 23.5.2026).

- [19] Robert C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017.
- [20] Microsoft Learn, *Enable Cross-Origin Requests (CORS) in ASP.NET Core*, 2024, URL: <https://learn.microsoft.com/en-us/aspnet/core/security/cors?view=aspnetcore-7.0> (accesat în 23.5.2026).
- [21] Microsoft Learn, *Handle errors in ASP.NET Core*, 2025, URL: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/error-handling?view=aspnetcore-9.0> (accesat în 23.5.2026).
- [22] Npgsql Development Team, *Npgsql Entity Framework Core Provider*, 2026, URL: <https://www.npgsql.org/efcore/?tabs=aspnet> (accesat în 23.5.2026).
- [23] OpenAI, *Chat Completions*, 2026, URL: <https://platform.openai.com/docs/api-reference/chat/commands> (accesat în 23.5.2026).
- [24] OpenAI, *Structured model outputs*, 2026, URL: <https://platform.openai.com/docs/guides/structured-outputs?api-mode=chat> (accesat în 23.5.2026).
- [25] OWASP Foundation, *Authentication Cheat Sheet*, 2026, URL: [https://cheatsheetseries.owasp.org/cheatsheets/Authentication\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html) (accesat în 23.5.2026).
- [26] OWASP Foundation, *Multifactor Authentication Cheat Sheet*, 2026, URL: [https://cheatsheetseries.owasp.org/cheatsheets/Multifactor\\_Authentication\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Multifactor_Authentication_Cheat_Sheet.html) (accesat în 23.5.2026).
- [27] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2026, URL: <https://www.postgresql.org/docs/16/> (accesat în 23.5.2026).
- [28] Yichi Shen et al., „UICoder: Finetuning Large Language Models to Generate User Interface Code through Automated Feedback”, în *arXiv preprint arXiv:2406.07739* (2024), URL: <https://arxiv.org/abs/2406.07739> (accesat în 23.5.2026).
- [29] Ben Shneiderman, „Direct Manipulation: A Step Beyond Programming Languages”, în *Computer* 16.8 (1983), pp. 57–69, DOI: 10.1109/MC.1983.1654471, URL: <http://doi.ieeecomputersociety.org/10.1109/MC.1983.1654471> (accesat în 25.5.2026).
- [30] Webflow, *Visual website builder*, 2026, URL: <https://webflow.com/> (accesat în 23.5.2026).
- [31] Austin Wright et al., *JSON Schema Draft 2020-12*, 2022, URL: <https://json-schema.org/draft/2020-12> (accesat în 23.5.2026).
- [32] Jinyu Xu et al., „LaTCoder: Converting Webpage Design to Code with Layout-as-Thought”, în *arXiv preprint arXiv:2508.03560* (2025), URL: <https://arxiv.org/abs/2508.03560> (accesat în 23.5.2026).